Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Eneida Balliu

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the System Administration Module through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill administrative requirements.

Technical Scope

This document focuses on the interaction between:

1. Actors: The System Administrator.
2. Boundary Objects: User Interfaces (UIs) used for data input and display.
3. Control Objects: Controllers that orchestrate business logic and validation.
4. Entity Objects: The database layer including User Accounts, System Logs, and Resources.

System Interaction Standards

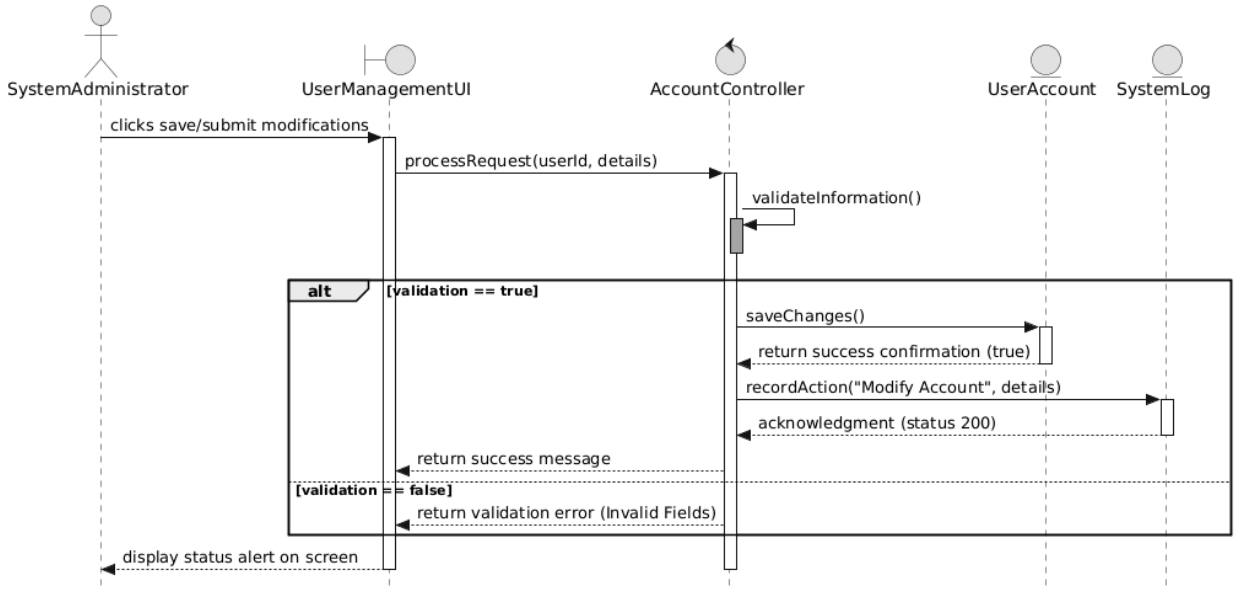
To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

1. Strict UML Notation: All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
2. Controller-Centric Logic: To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
3. Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
4. Automated Audit Integration: Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger to the SystemLog entity, satisfying non-functional auditing requirements.
5. Linear Success Flow: The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

UC_40: Manage User Accounts

Focus: Account Lifecycle

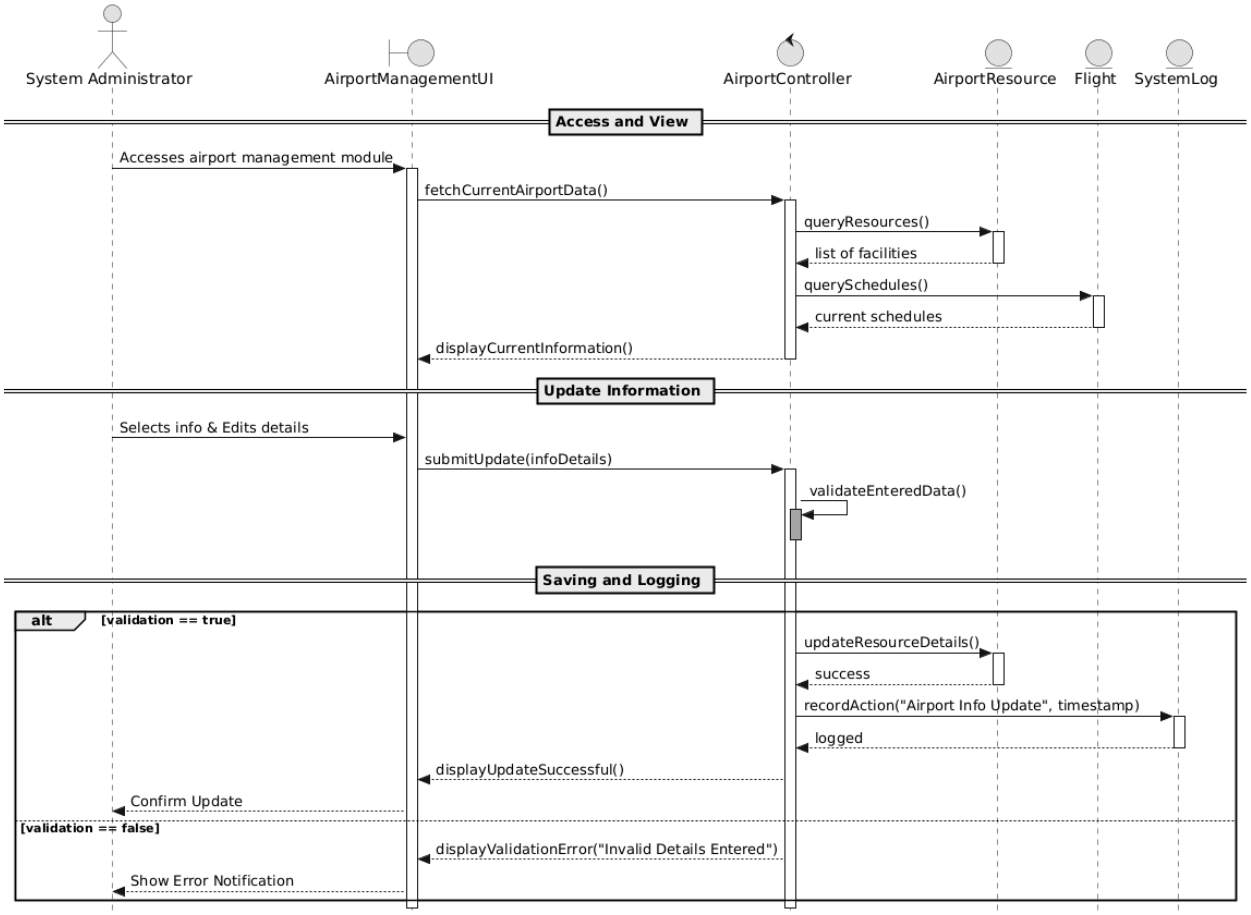
- Description: This diagram illustrates the standard process for creating, updating, or deleting user accounts. It demonstrates the interaction between the Administrator and the core identity management system.
- Key Logic: The AccountController performs a self-validation check on user data before committing any changes to the UserAccount entity.
- Compliance: The flow concludes with a mandatory update to the SystemLog, ensuring all administrative changes are audited.



UC_41: Update Airport Information

Focus: Resource and Schedule Maintenance

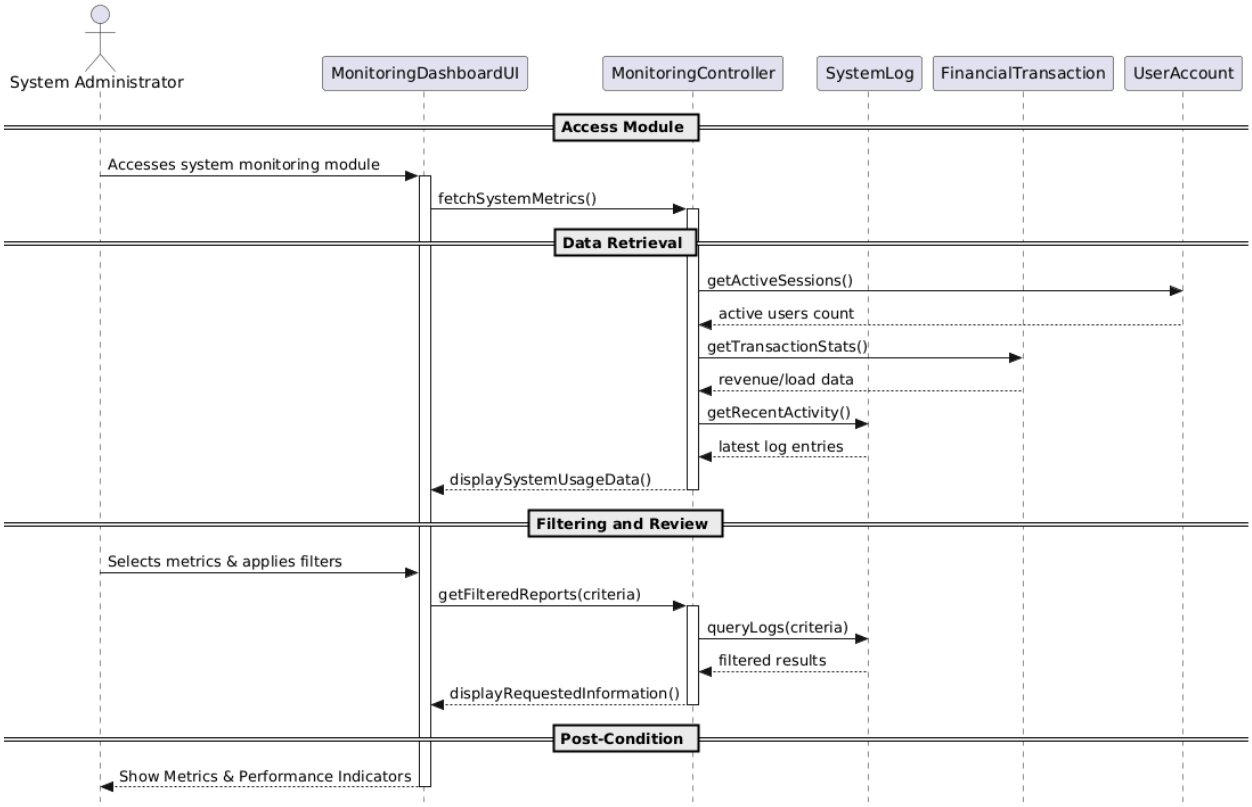
- Description: This sequence shows how the system updates static and dynamic airport data, specifically facilities and flight schedules.
- Key Logic: The diagram highlights the system's ability to interact with multiple entities (AirportResource and Flight) through a single administrative request.
- Compliance: Data integrity is maintained by validating input at the controller level before updating the database.



UC_42: Monitor System Usage

Focus: Real-time Performance Visualization

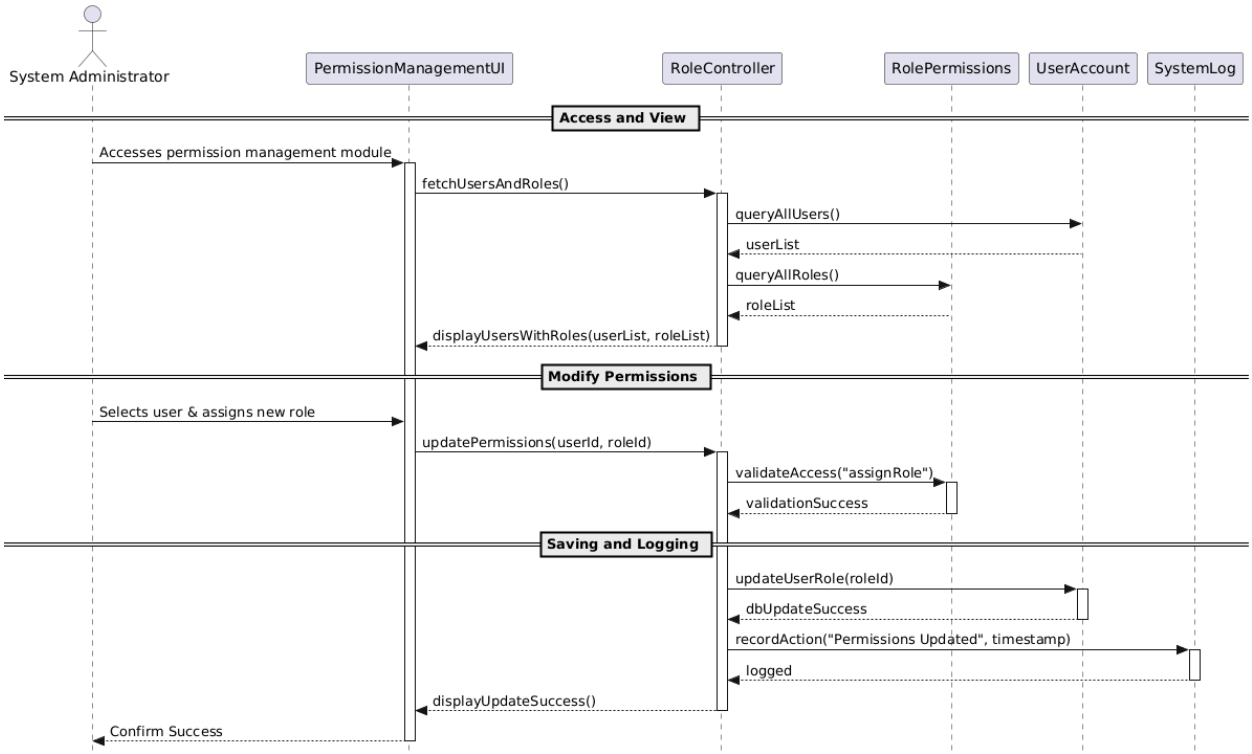
- Description: This diagram represents a "Read-Only" operation where the administrator retrieves live system performance indicators.
- Key Logic: The MonitoringController acts as an aggregator, pulling concurrent data from UserAccount (active users), FinancialTransaction (system load), and SystemLog (activity history).
- Compliance: It fulfills the requirement for high system availability by providing real-time data without modifying the database.



UC_43: Manage System Permissions

Focus: Role-Based Access Control (RBAC)

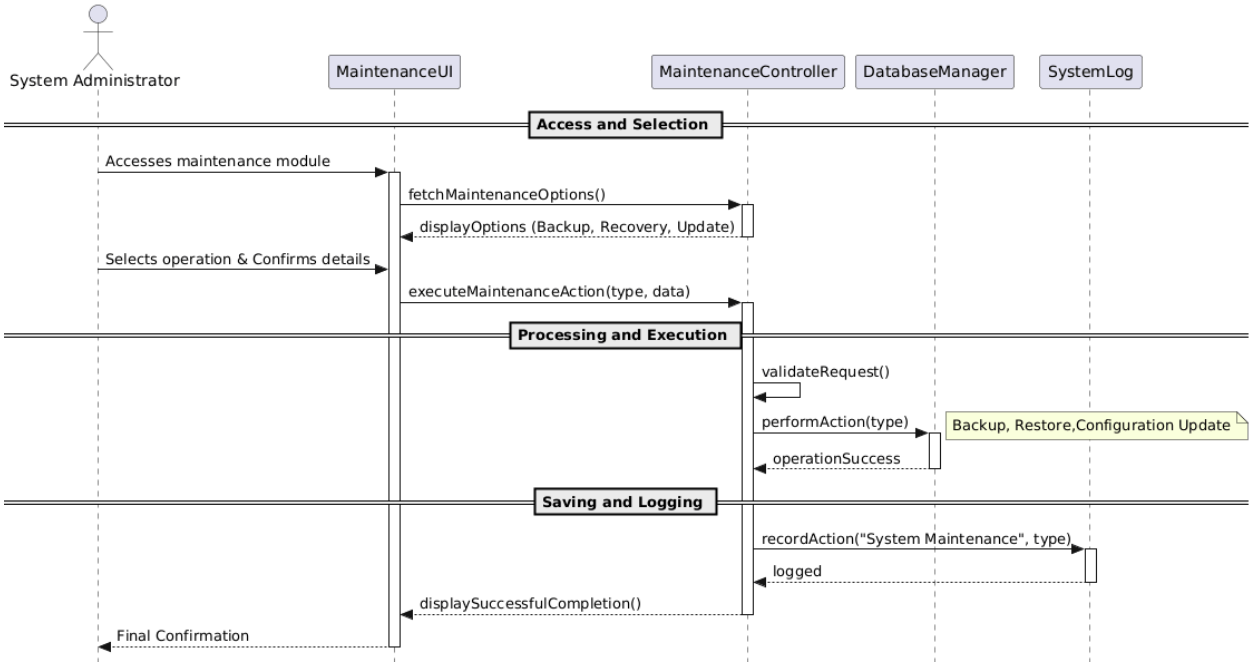
- Description: This sequence illustrates the process of assigning or modifying access rights for existing users.
- Key Logic: A specific security step is included where the RoleController calls validateAccess on the RolePermissions class to ensure the change is authorized and consistent with system rules.
- Compliance: Strict adherence to RBAC standards is demonstrated by separating the "Role" logic from the "User" data.



UC_44: Perform System Maintenance

Focus: System Health and Data Safety

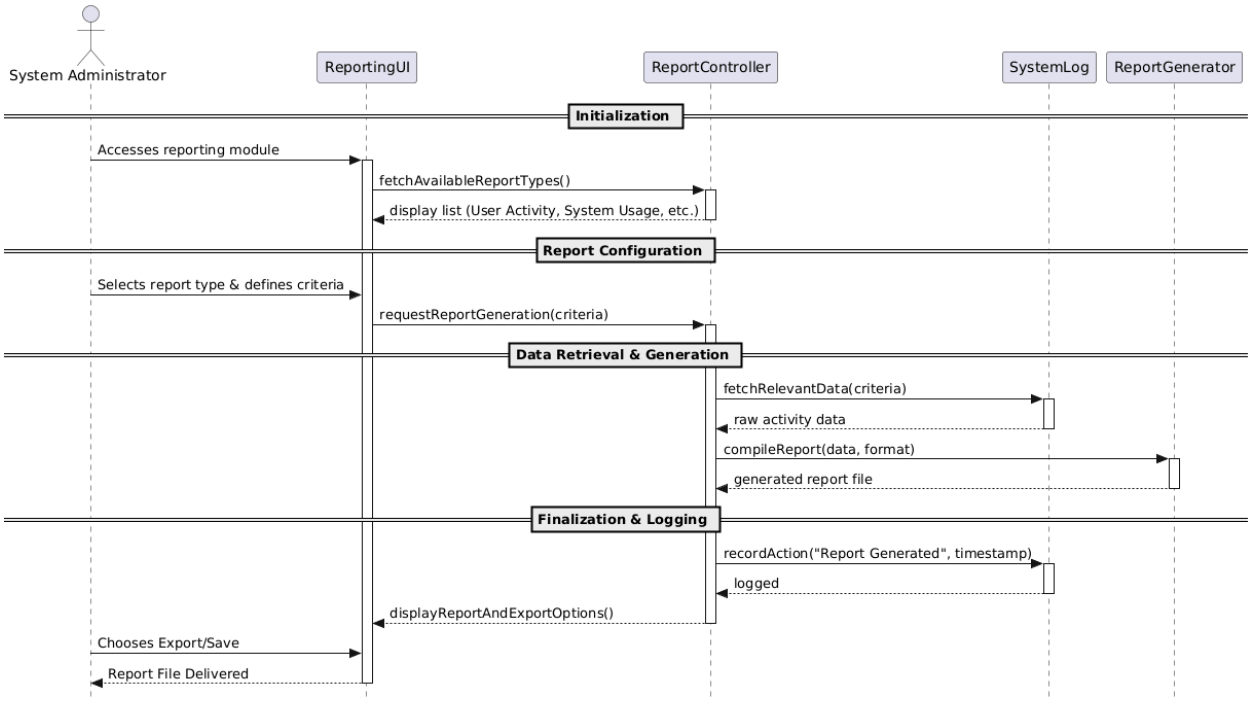
- Description: This diagram covers critical maintenance tasks: data backup, recovery, and system configuration updates.
- Key Logic: The interaction with a specialized DatabaseManager shows that these operations are performed at a system-wide level rather than a user level.
- Compliance: It emphasizes data integrity and disaster recovery protocols, recording each maintenance event in the audit logs.



UC_45: Generate System Activity Reports

Focus: Data Analysis and Exporting

- Description: This final sequence shows how system data is processed into formal reports (PDF/CSV) for administrative review.
- Key Logic: The diagram introduces a ReportGenerator utility class, separating the complex task of document compilation from standard data retrieval.
- Compliance: It satisfies the non-functional requirement for reporting and exporting, providing a documented trail of system activity.



Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Alvi Elezi

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the Flight Operations Module through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill administrative requirements.

Technical Scope

This document focuses on the interaction between:

1. Actors: Airport Staff and Passenger.
2. Boundary Objects: User Interfaces (UIs) used for data input and display.
3. Control Objects: Controllers that orchestrate business logic and validation.
4. Entity Objects: The database layer including User Accounts, System Logs, and Resources.

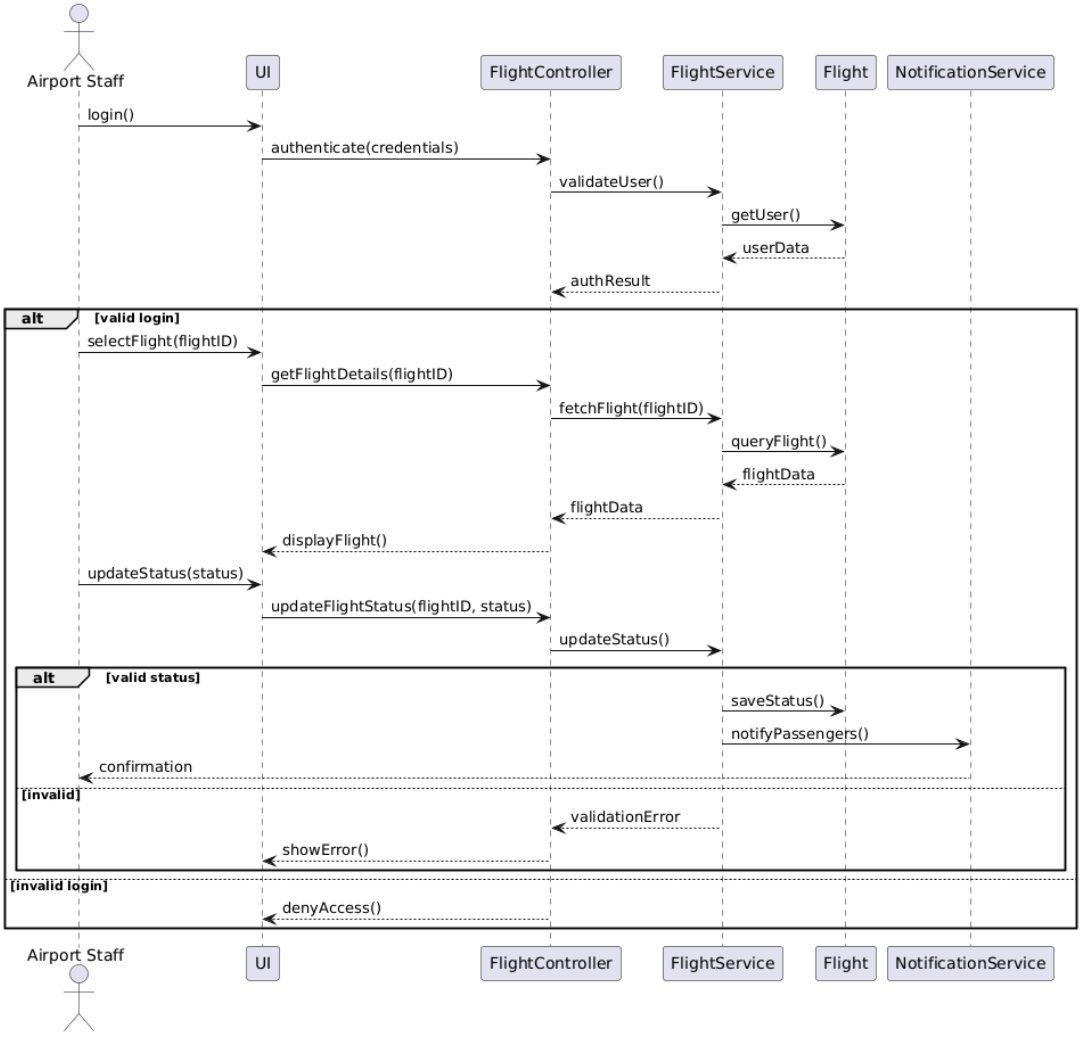
System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

1. Strict UML Notation: All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
2. Controller-Centric Logic: To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
3. Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
4. Automated Audit Integration: Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger to the SystemLog entity, satisfying non-functional auditing requirements.
5. Linear Success Flow: The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

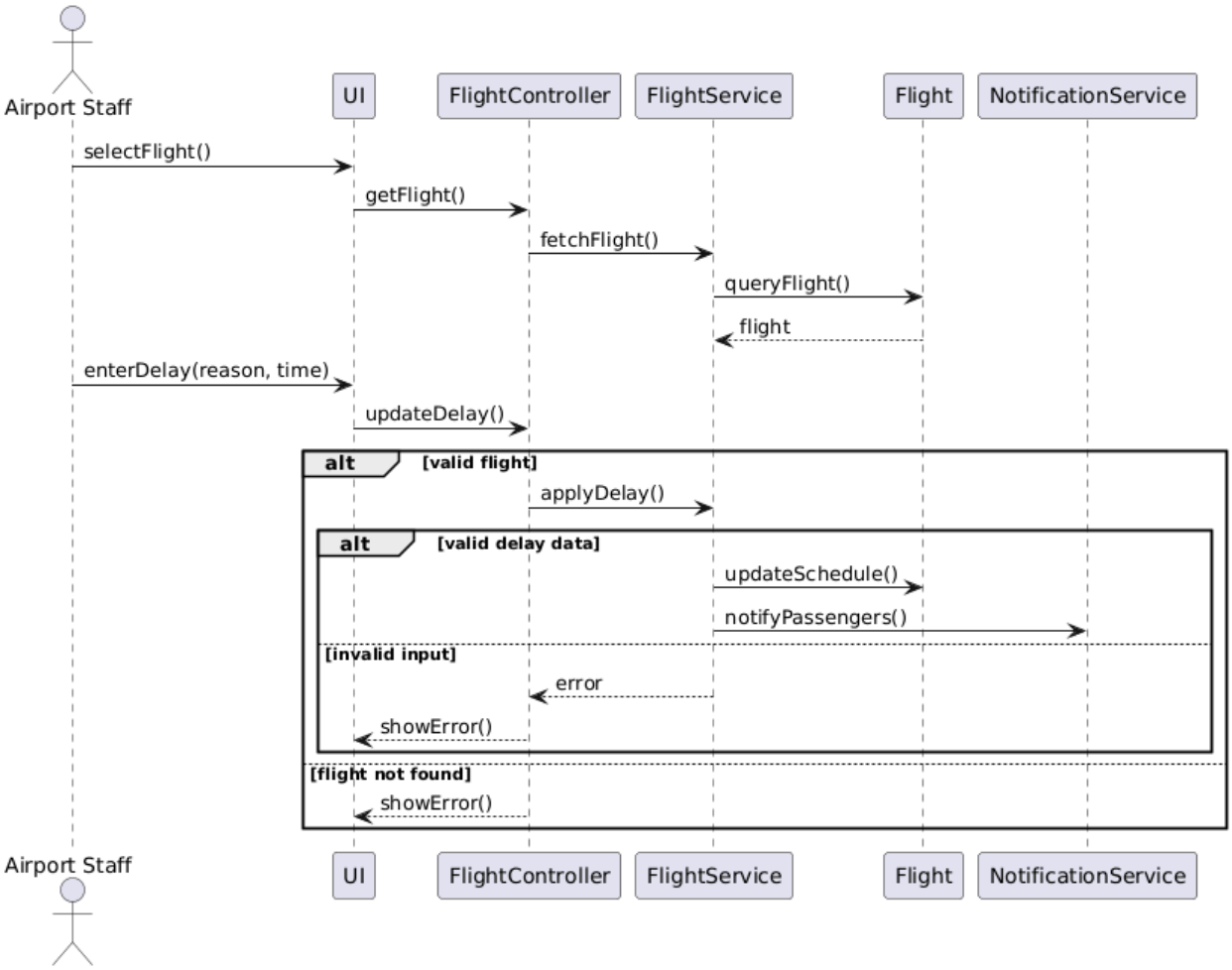
1.Update flight Status

- **Focus:** Flight Status Management
- **Description:** This sequence illustrates how airport staff update the operational status of a flight and propagate the change across the system.
- **Key Logic:** The controller validates the update request, retrieves the flight entity, applies the new status, and triggers notifications to passengers.
- **Compliance:** Data consistency is ensured through validation before persistence, and system-wide synchronization is achieved via notification services.



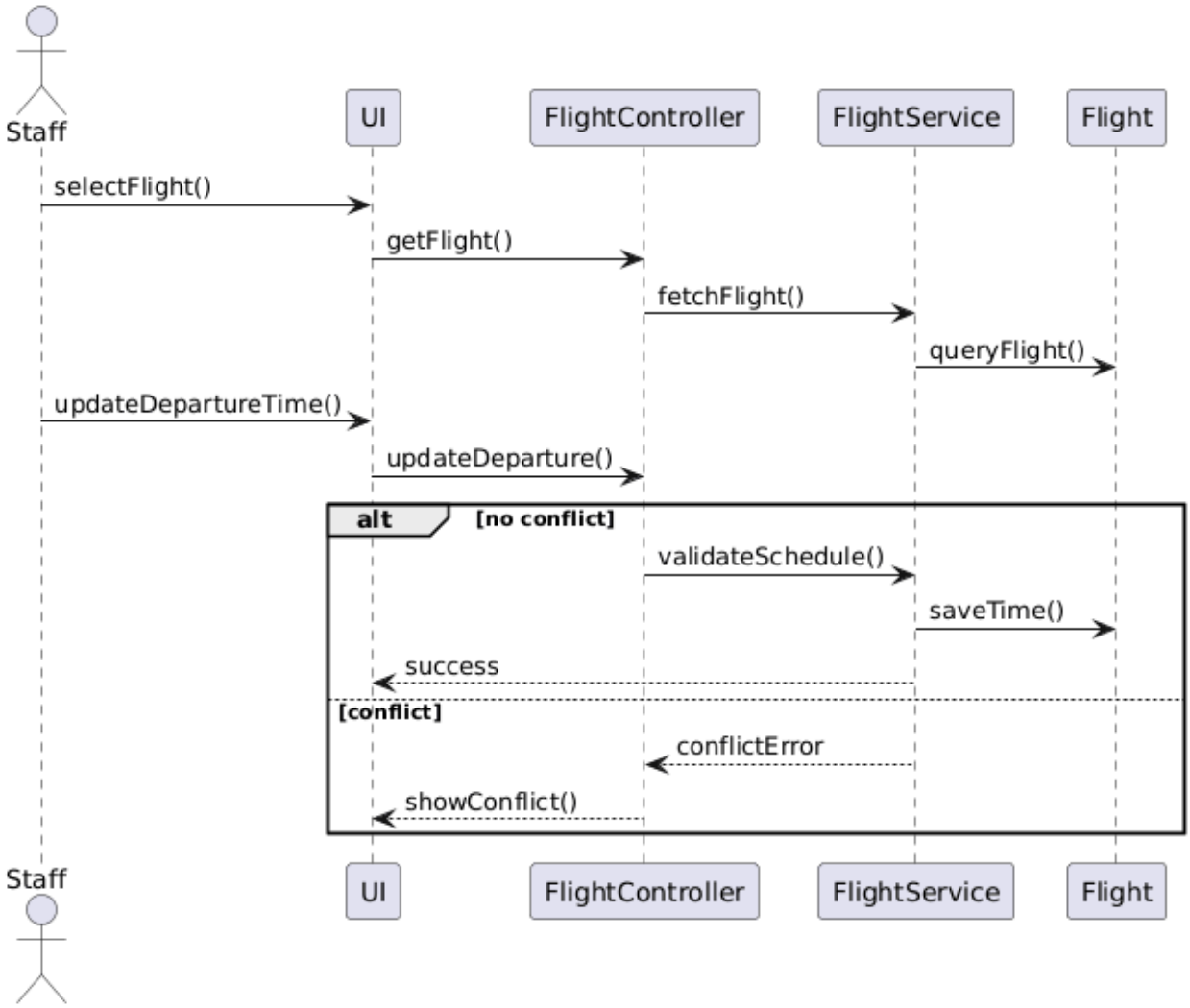
2.Manage Flight Delays

- **Focus:** Delay Handling and Schedule Adjustment
- **Description:** This sequence demonstrates how delays are recorded, processed, and reflected in the flight schedule.
- **Key Logic:** The system validates delay input, updates the flight schedule, and automatically notifies affected passengers.
- **Compliance:** Input validation at the controller level prevents inconsistent scheduling, ensuring reliable operational update**s**



3. Update departure time

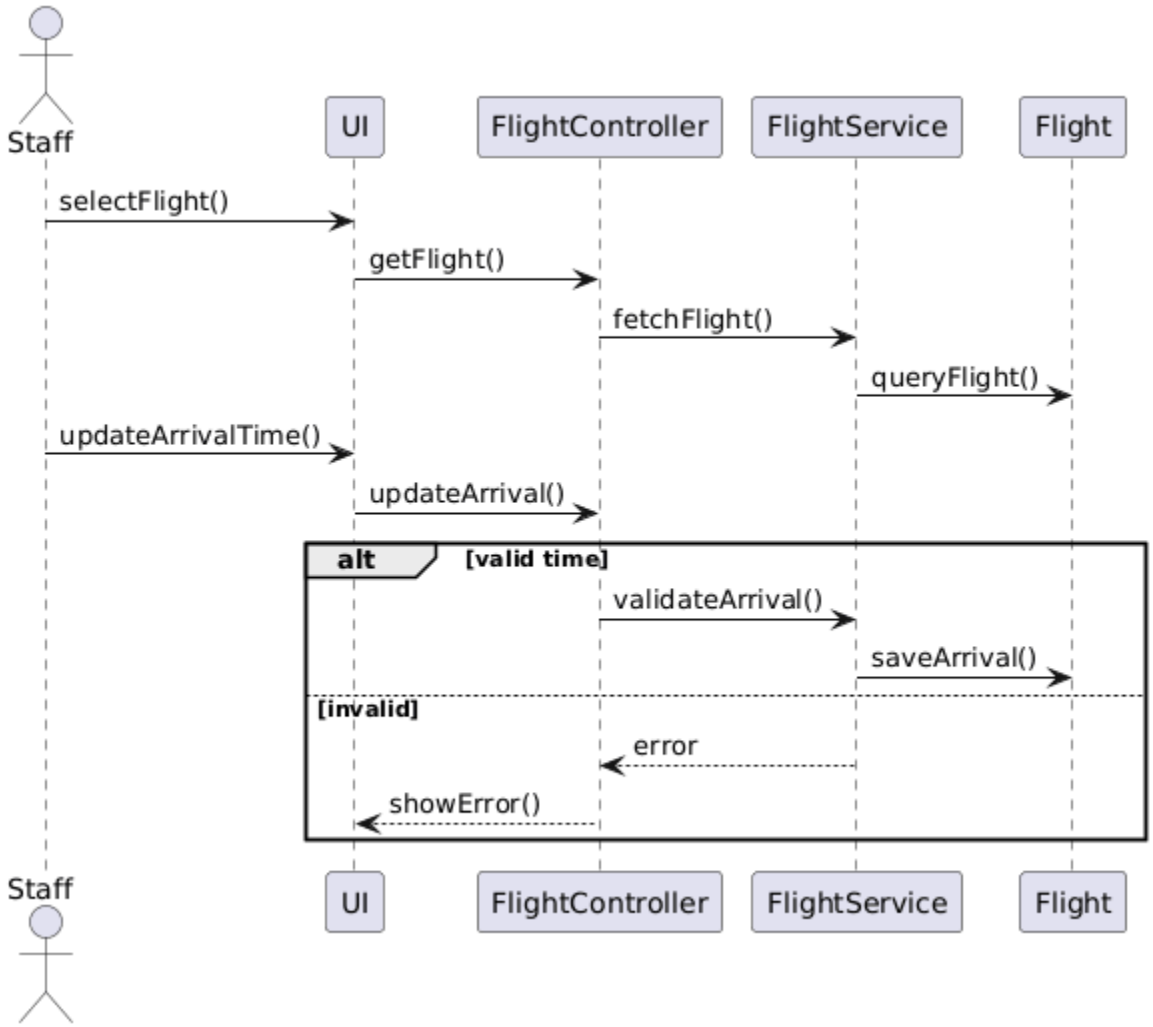
- **Focus:** Departure Scheduling
- **Description:** This sequence shows how staff modify the scheduled departure time of a flight.
- **Key Logic:** The controller verifies scheduling constraints (such as runway availability) before committing updates to the system.
- **Compliance:** Conflict detection mechanisms ensure that updated schedules do not violate operational constraints.



4.Update arrival time

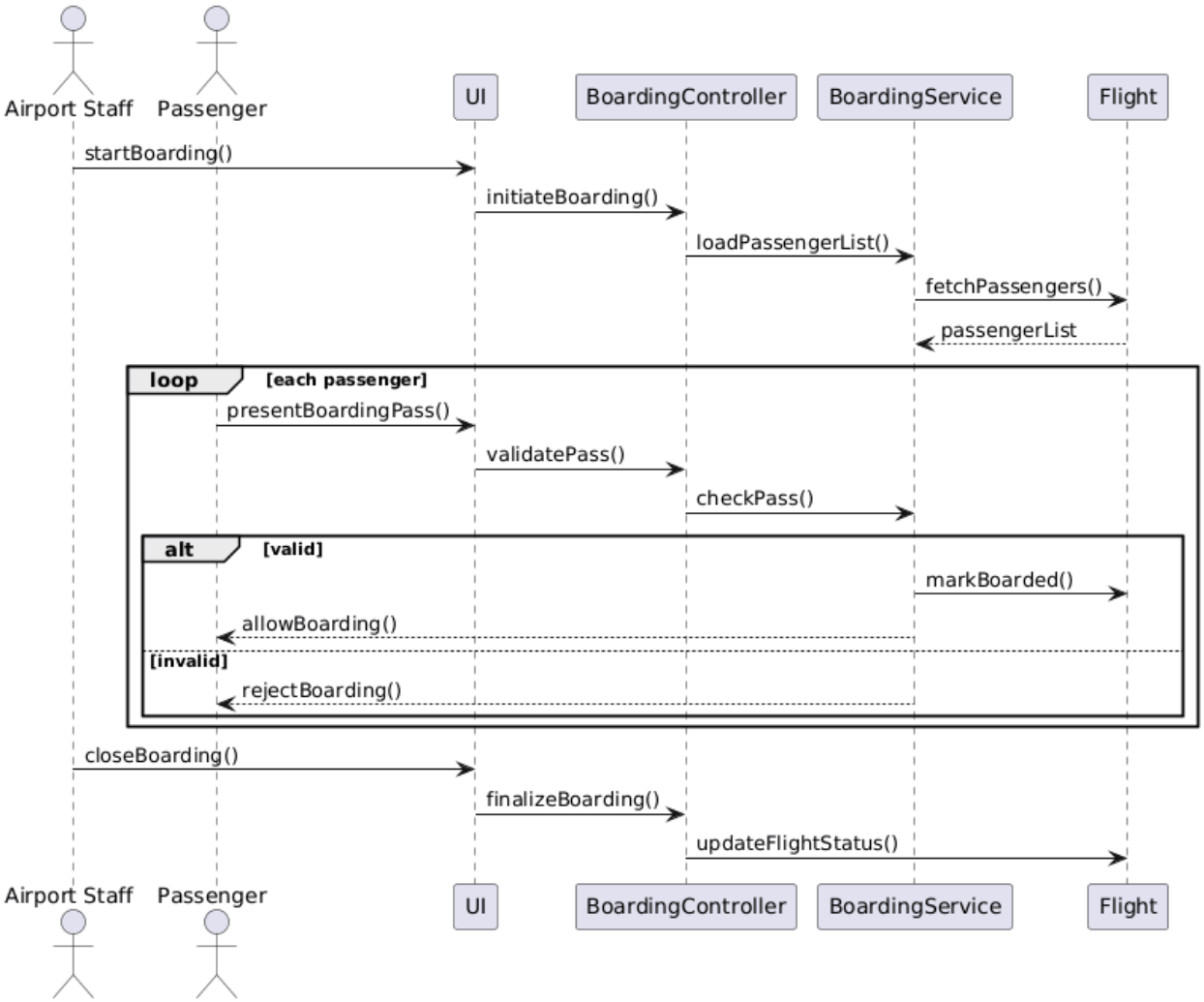
- **Focus:** Arrival Time Synchronization
- **Description:** This sequence represents how arrival time updates are processed and reflected across the system.
- **Key Logic:** The system validates the new arrival time and updates the flight entity while maintaining synchronization with operational displays.
- **Compliance:** Validation ensures logical consistency between departure and arrival times.

.



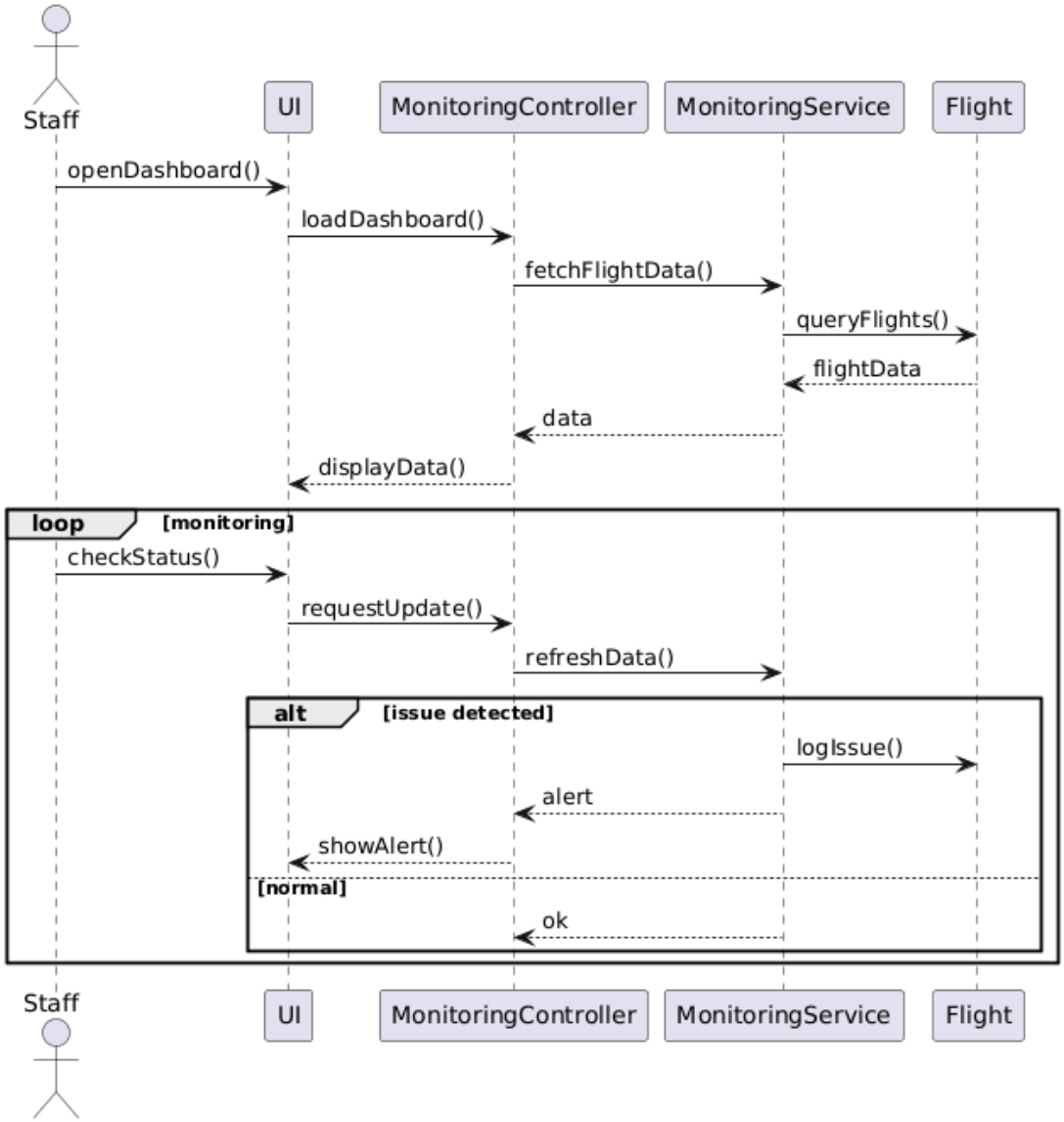
5.Coordinate boarding process

- **Focus**: Passenger Boarding Control
- **Description**: This sequence details how the boarding process is initiated, managed, and completed by airport staff.
- **Key Logic**: The system iteratively validates boarding passes, updates passenger status, and tracks boarding progress in real time.
- **Compliance**: Each boarding action updates the system state, ensuring accurate passenger tracking and operational visibility.



6.Monitor Flight Operations

- **Focus:** Real-Time Operational Monitoring
- **Description:** This sequence shows how airport staff monitor flight operations through a centralized dashboard.
- **Key Logic:** The system continuously retrieves and refreshes flight data, detects anomalies, and generates alerts when issues arise.
- **Compliance:** Continuous data synchronization ensures that operational decisions are based on up-to-date information.



Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: *Ajla Alcani*

Course: *Software Modeling and Design*

Date: *April 10, 2026*

Document Objective

The primary objective of this document is to detail the dynamic behavior of the ***Human Resources and Customer Support Modules*** *through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill system requirements.*

Technical Scope

This document focuses on the interaction between:

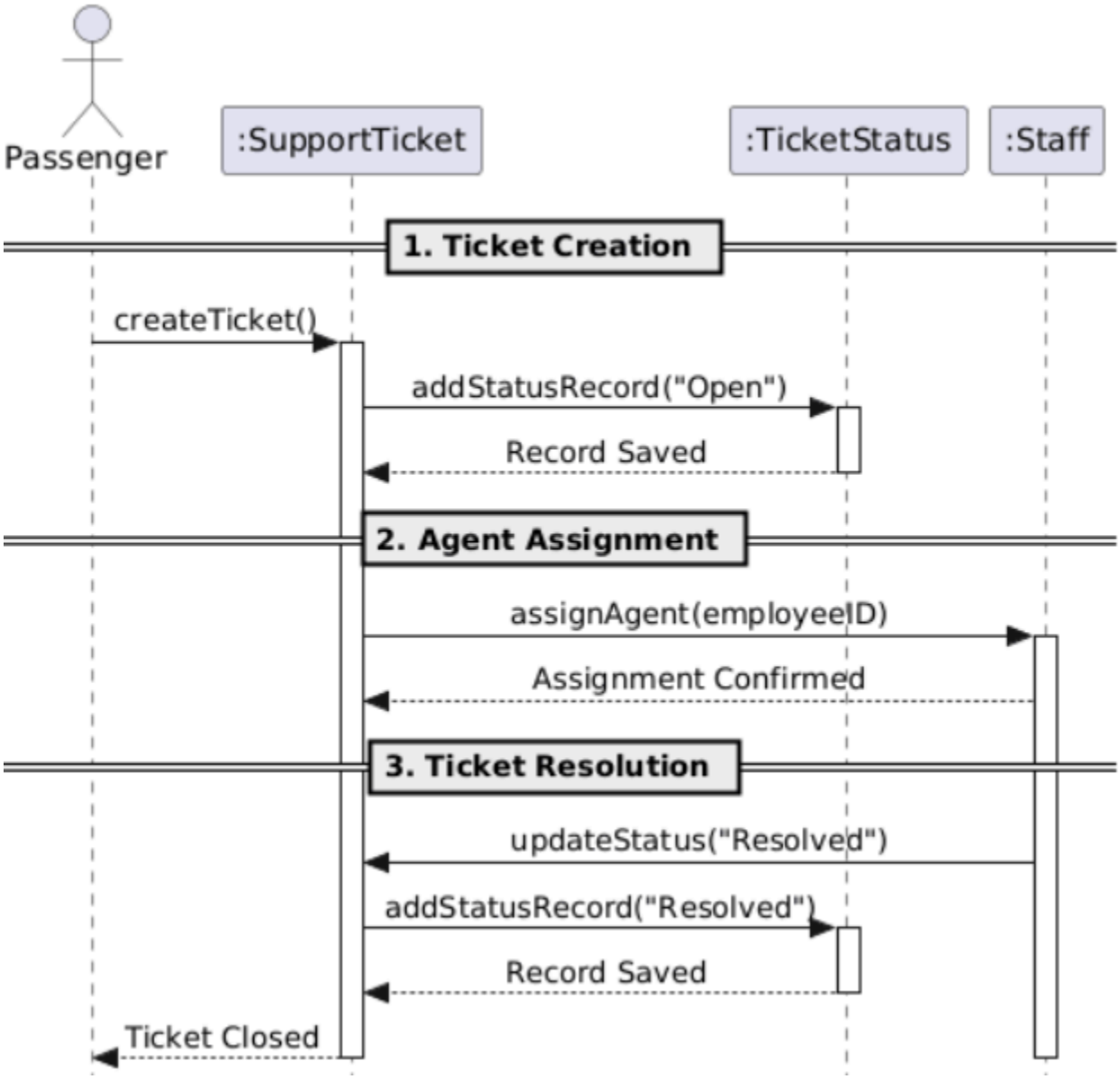
1. ***Actors:*** *HR Admin, Employee (Staff), and Passenger.*
2. ***Boundary Objects:*** *User Interfaces (UIs) used for data input and display (e.g., Chat UI, Ticketing Dashboard).*
3. ***Control Objects:*** *Controllers that orchestrate business logic and validation for staff and support operations.*
4. ***Entity Objects:*** *The database layer including SupportTicket, TicketStatus, Staff, AttendanceRecord, Payroll, and ChatSession.*

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

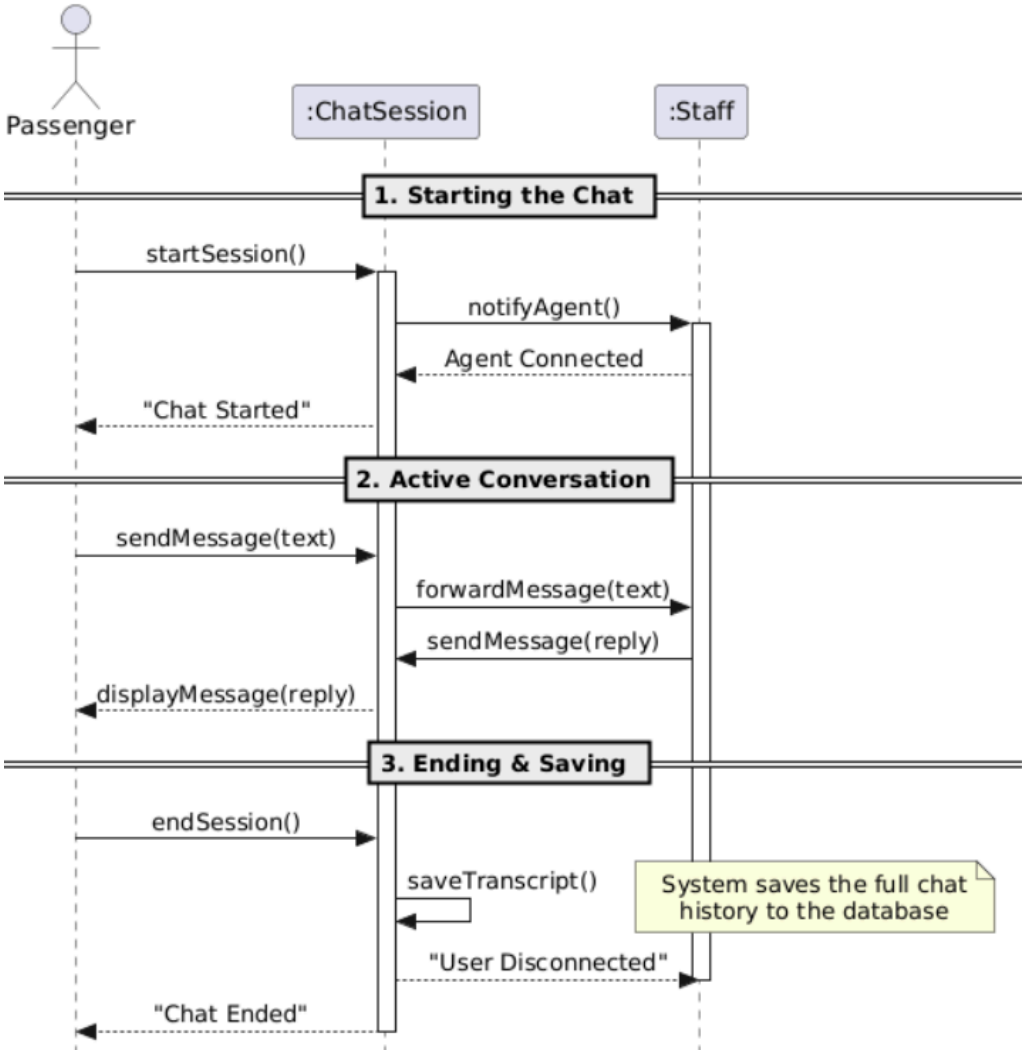
1. ***Strict UML Notation:*** *All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).*
2. ***Controller-Centric Logic:*** *To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.*
3. ***Entity-Layer Persistence:*** *Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.*
4. ***Automated Audit Integration:*** *Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger, satisfying non-functional auditing requirements.*
5. ***Linear Success Flow:*** *The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.*

1. Submit Support Ticket



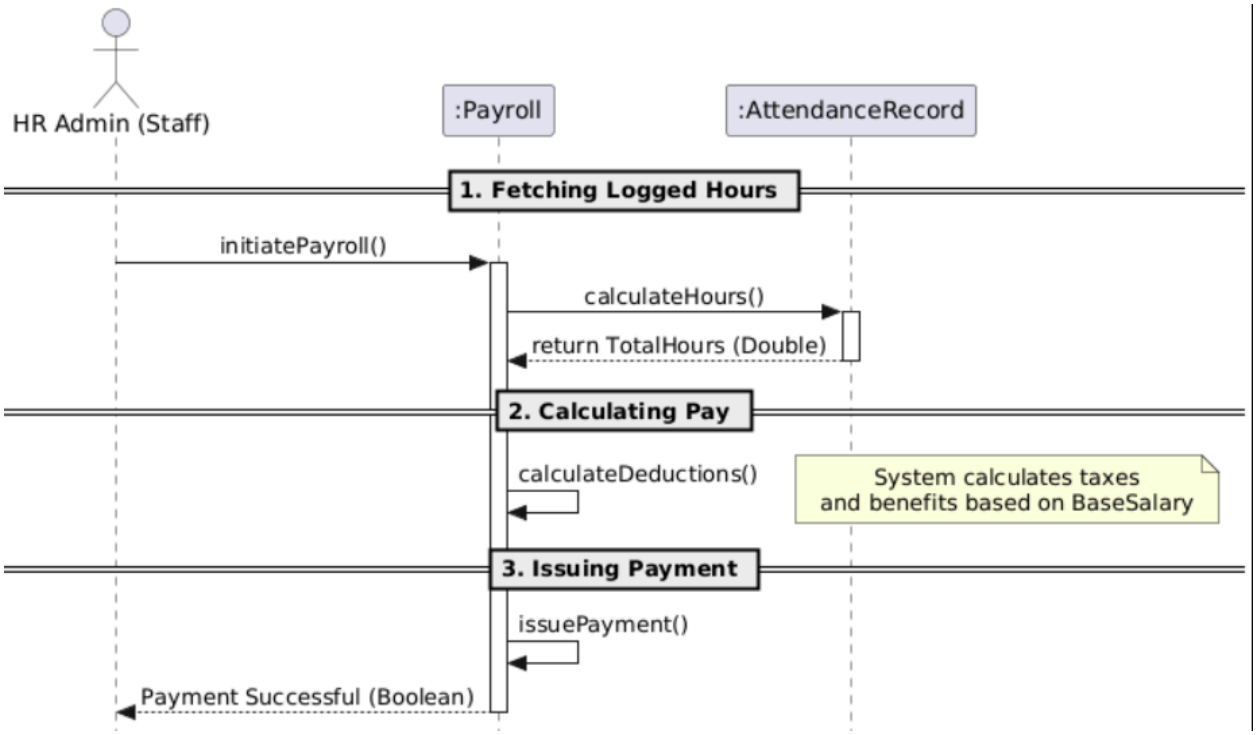
- **Focus:** Issue Tracking and Resolution
- **Description:** This sequence illustrates how a passenger creates a support ticket, how it is assigned to an agent, and resolved.
- **Key Logic:** The system logs the initial "Open" state in the TicketStatus entity, assigns a Staff agent, and updates the status to "Resolved" upon completion.
- **Compliance:** All state changes are immediately recorded in the status entity to maintain a complete and accurate audit trail of the ticket's lifecycle.

2. Live Chat Session



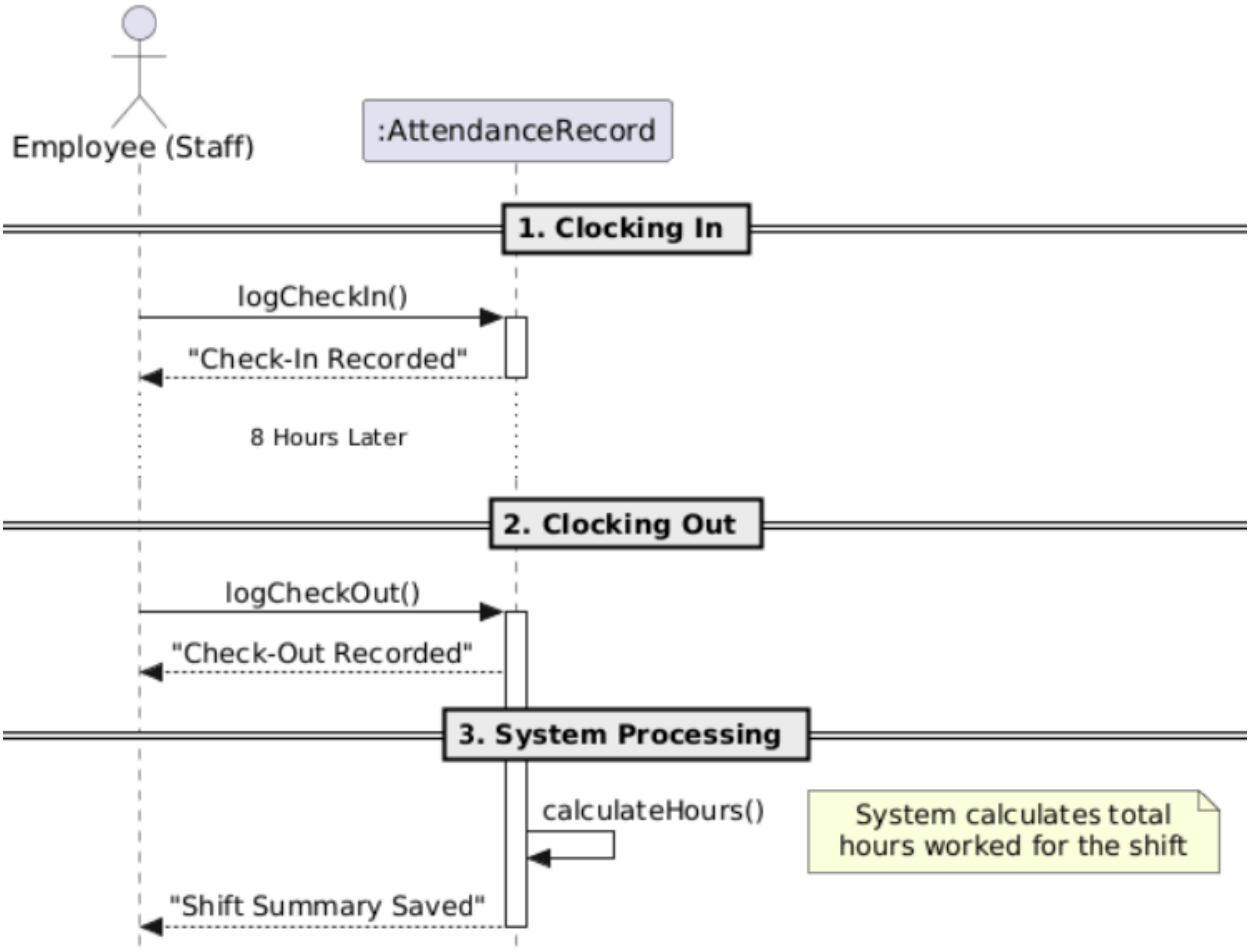
Focus: *Real-Time Customer Assistance*
Description: *This diagram covers the synchronous, real-time communication between a Passenger and a Staff agent through the system.*
Key Logic: *The ChatSession acts as the intermediary, forwarding messages back and forth. Upon disconnection, it triggers a self-message to save the transcript.*
Compliance: *The system automatically fulfills data retention requirements by saving the full chat history to the database the moment the session ends.*

3. Process Payroll



Focus: *Salary Calculation and Disbursement*
Description: *This sequence demonstrates the internal process of calculating an employee's pay based on their logged hours and issuing the final payment.*
Key Logic: *The Payroll entity interacts with the AttendanceRecord to fetch total hours, executes an internal calculation for deductions, and issues a success confirmation.*
Compliance: *Financial calculations and raw working hours are strictly separated into different entities to maintain data integrity before issuing any payments.*

4. Log Staff Attendance



Focus: *Employee Time Tracking*
Description: *This sequence shows an employee clocking in at the start of their shift, clocking out, and the system processing their total duration.*
Key Logic: *The AttendanceRecord captures exact timestamps for check-in and check-out, then executes an internal system process to calculate the total hours worked.*
Compliance: *Automated time calculations at the entity level prevent manual entry errors, ensuring perfectly accurate data is passed to the payroll module.*

Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Andri Mucllari

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

TThis document presents the **dynamic behavior** of the **Financial and Billing Management Module** through a series of **Sequence Diagrams**. This diagrams illustrate **how objects interact over time** to handle payment processing, invoice generation, and financial record management.

Technical Scope

This document focuses on the interaction between:

1. **Actors:** Finance Staff and Customers.
2. **Boundary Objects:** User Interfaces for payment entry, invoice review, and report generation.
3. **Control Objects:** Controllers that manage transaction validation, invoice computation, and payment confirmation.
4. **Entity Objects:** Database entities including Customer Accounts, Payment Records, Invoices, and System Logs.

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document follows these architectural rules:

1. **Strict UML Notation:** All models utilize standard UML symbols. Solid lines represent synchronous requests, while dashed lines indicate responses or returned data.
2. **Controller-Centric Logic:** To keep the UI lightweight, all business rules, transaction processing, and validation are handled within Controllers.
3. **Entity-Layer Persistence:** Communication with the database occurs only through Entity objects, ensuring a clean separation of logic and data storage.
4. **Automated Audit Integration:** Any action affecting system state (Create, Update, Delete) triggers a corresponding entry in the SystemLog entity to satisfy auditing requirements.

Payment Processing

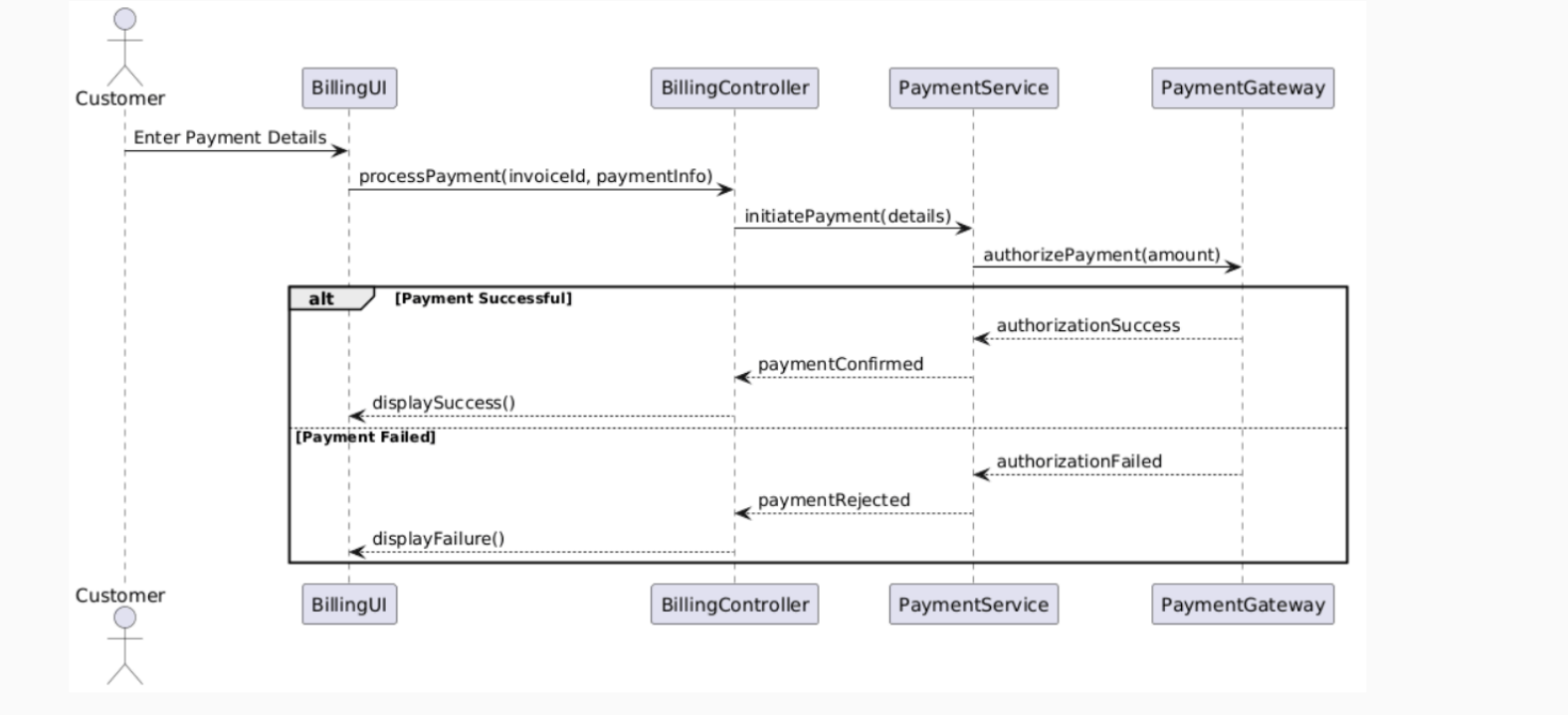
Objective:

To model how an invoice is generated and delivered to a customer

This diagram traces the end-to-end payment journey from a customer entering payment details through to success or failure outcomes. The flow begins with the Customer entering payment details into the billingUI.

The billingController then calls processPayment(invoiced, paymentInfo), which triggers the PaymentService to communicate with the PaymentGateway.

The gateway attempts authorizePayment(amount), returning either authorizationSuccess or authorizationFailed. On success, paymentConfirmed leads to displaySuccess(). On failure, paymentRejected leads to displayFailure.

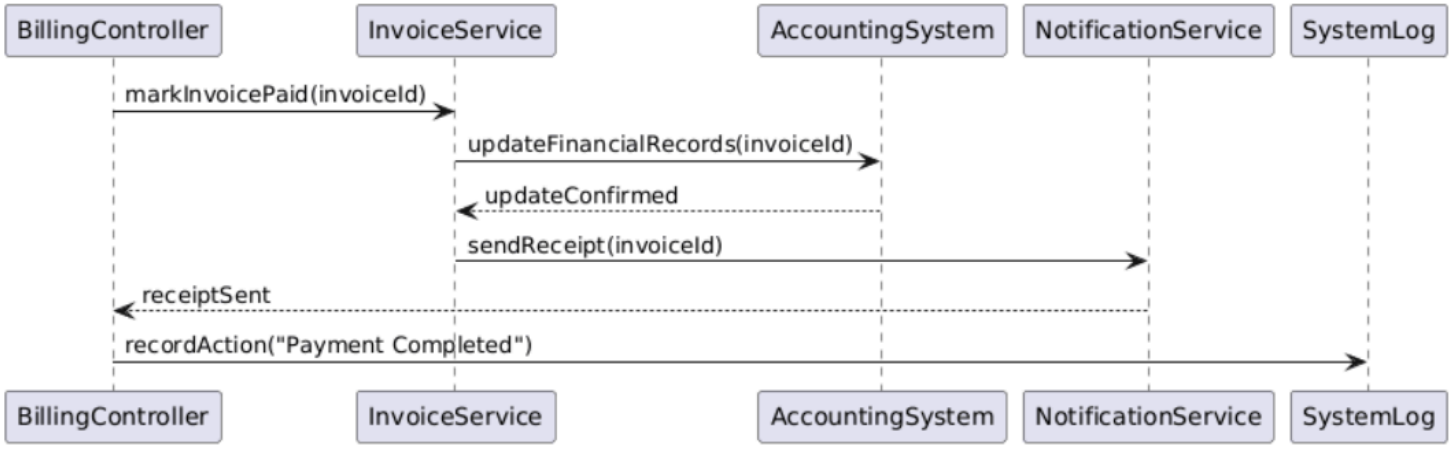


Payment Receipt & Financial Update

Objective:

To represent how the system updates financial records and sends a payment receipt after a successful transaction.

This diagram illustrates the post-payment workflow that occurs after a customer successfully pays. The BillingController coordinates with the system to complete the financial and customer notification.



Technical Design: Sequence Model Specifications

Prepared by: *Betül Kaan*

Course: *Software Modeling and Design*

Date: *April 10, 2026*

Document Objective

The primary objective of this document is to detail the dynamic behavior of the Passenger Service Management Module through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill administrative requirements.

Technical Scope

This document focuses on the interaction between:

1. Actors: Passenger and AirportStaff (secondary, UC-06 only).

2. Boundary Objects: User interfaces used for data input and display — FlightSearchUI, DocumentUploadUI, CheckInUI, BoardingPassUI, GateInfoUI, BaggageTrackingUI, and NotificationSettingsUI.

3. Control Objects: Controllers that orchestrate business logic and validation for passenger service operations — System(present in all UCs), CheckIn (manages window validation and boarding pass generation in UC-03), and FlightNotification(manages preference registration and alert triggering in UC-07).

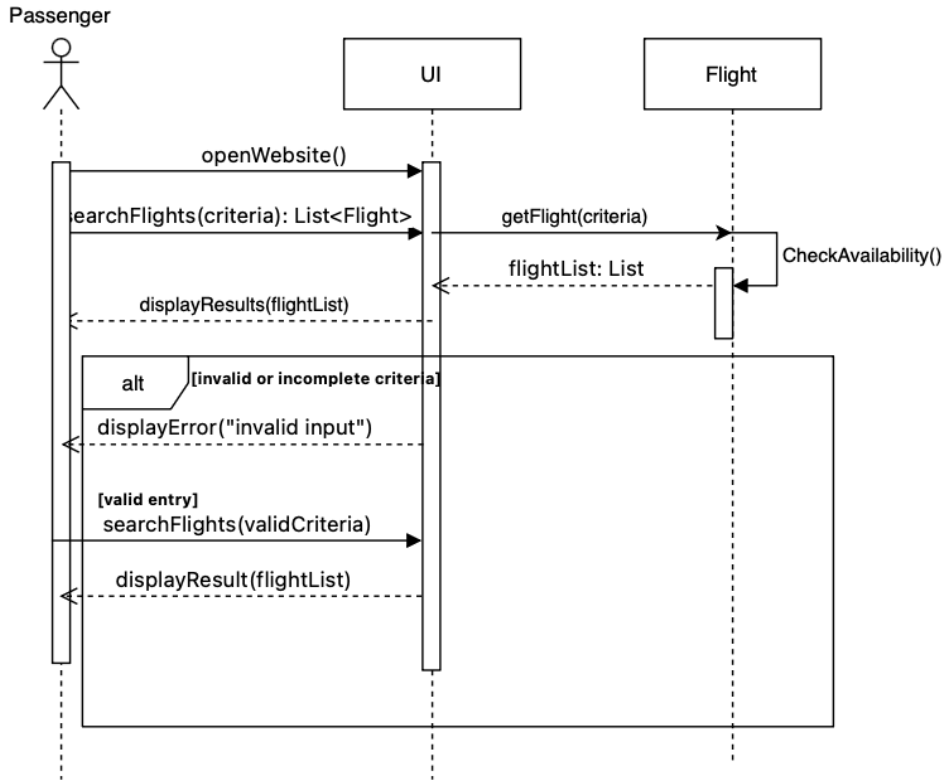
4. Entity Objects: The database layer including Flight, Booking, TravelDocument, BoardingPass, BaggageRecord, Gate, and FlightNotification.

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

6. Strict UML Notation: All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
7. Controller-Centric Logic: To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
8. Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
9. Automated Audit Integration: Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger to the SystemLog entity, satisfying non-functional auditing requirements.
10. Linear Success Flow: The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

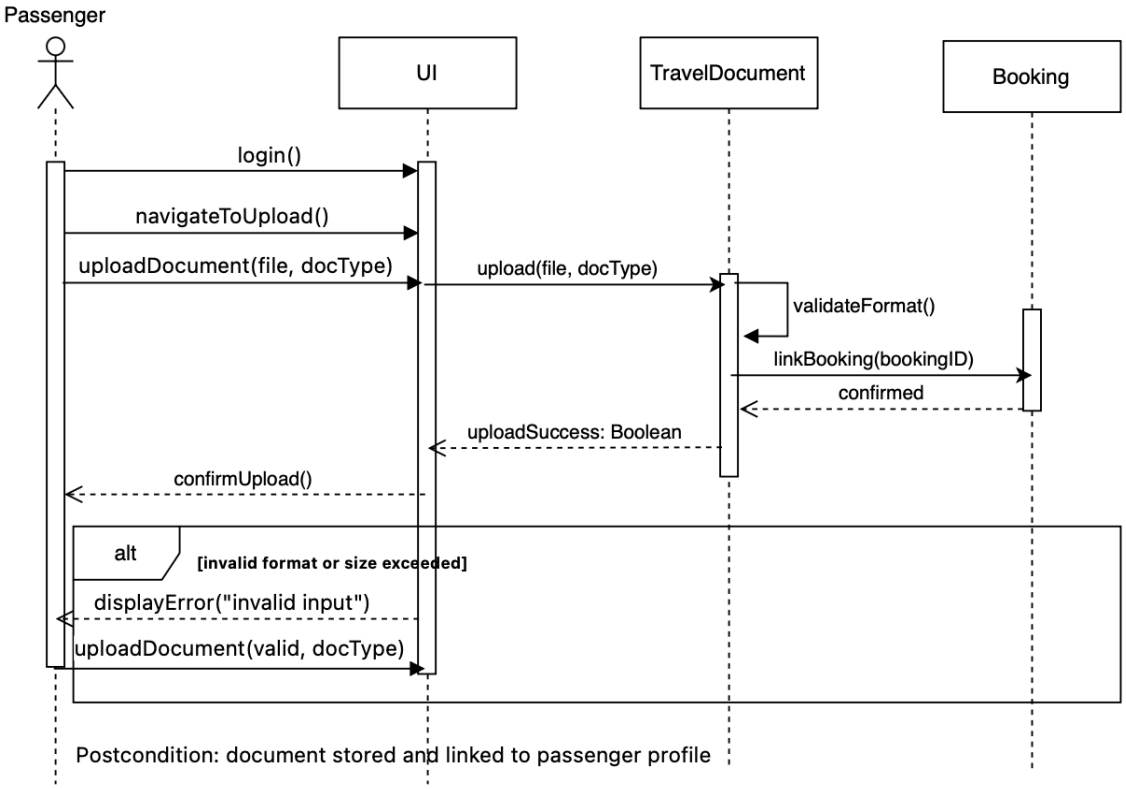
UC-01: View Flight Schedules



Postcondition: flight schedules displayed to passenger

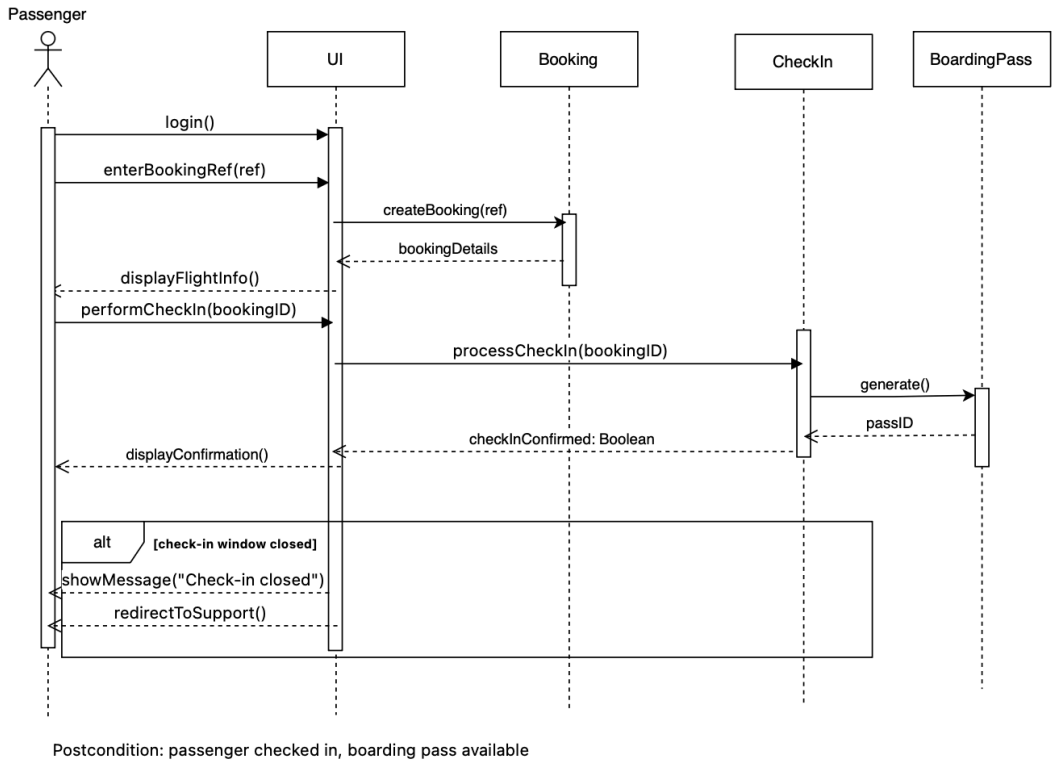
Focus: *Flight Search and Schedule Retrieval*
Description: *This sequence demonstrates how a passenger searches for available flights and receives matching schedule results from the system.*
Key Logic: *The system validates the search criteria, queries the Flight entity for matching records, and displays the results. An error message is shown if the input is invalid, prompting the passenger to retry.*
Compliance: *Input validation at the controller level prevents empty or malformed queries from reaching the data layer, ensuring accurate and reliable search results.*

UC-02: Upload Travel Documents



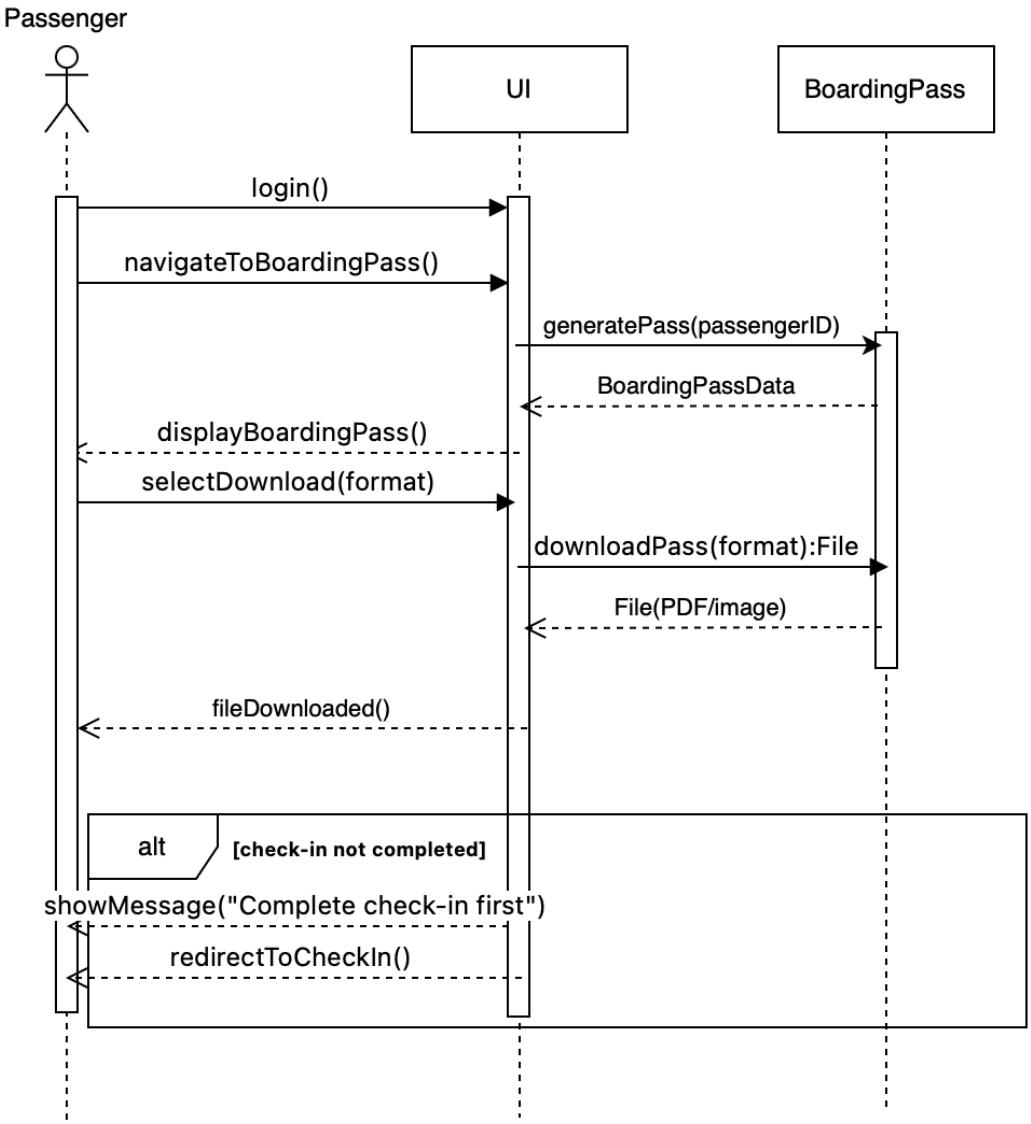
Focus: *Document Submission and Validation*
Description: *This sequence demonstrates how a passenger uploads travel documents that are validated and securely linked to their booking profile.*
Key Logic: *The system passes the uploaded file to TravelDocument, which validates the format and size before linking it to the corresponding Booking. A confirmation is returned upon success.*
Compliance: *Format and size validation at the entity level ensures only compliant documents are stored, protecting data integrity and meeting security requirements.*

UC-03: Perform Online Check-In



Focus: *Check-In Processing and Boarding Pass Generation*
Description: *This sequence demonstrates how a passenger completes online check-in by confirming their booking details, triggering the check-in process and boarding pass creation.*
Key Logic: *The CheckIn controller validates that the check-in window is open via isWindowOpen() before processing. Upon success, it calls generatePass() on BoardingPass. If the window is closed, the passenger is redirected to support.*
Compliance: *Window validation at the controller level prevents check-ins outside the allowed time frame, ensuring process integrity and consistent passenger handling.*

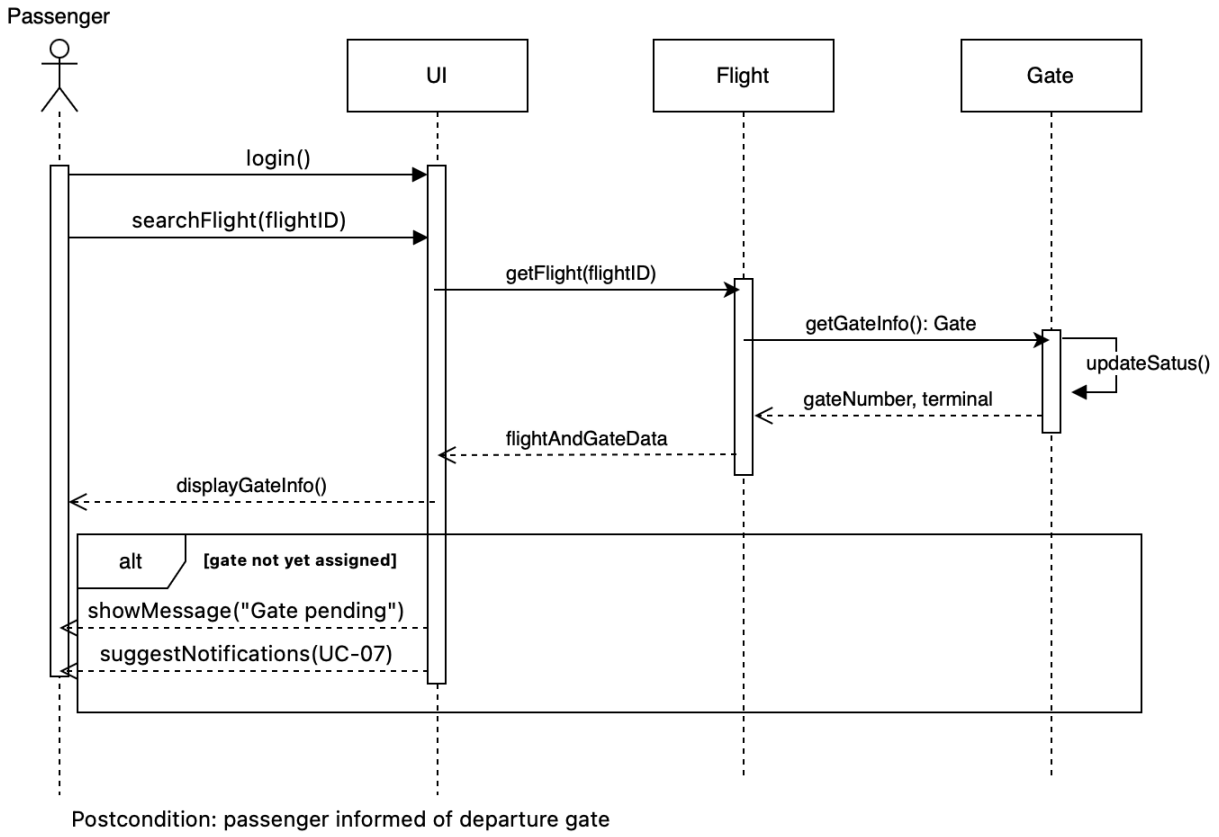
UC-04: Download Boarding Pass



Postcondition: passenger has dowloaded boarding pass

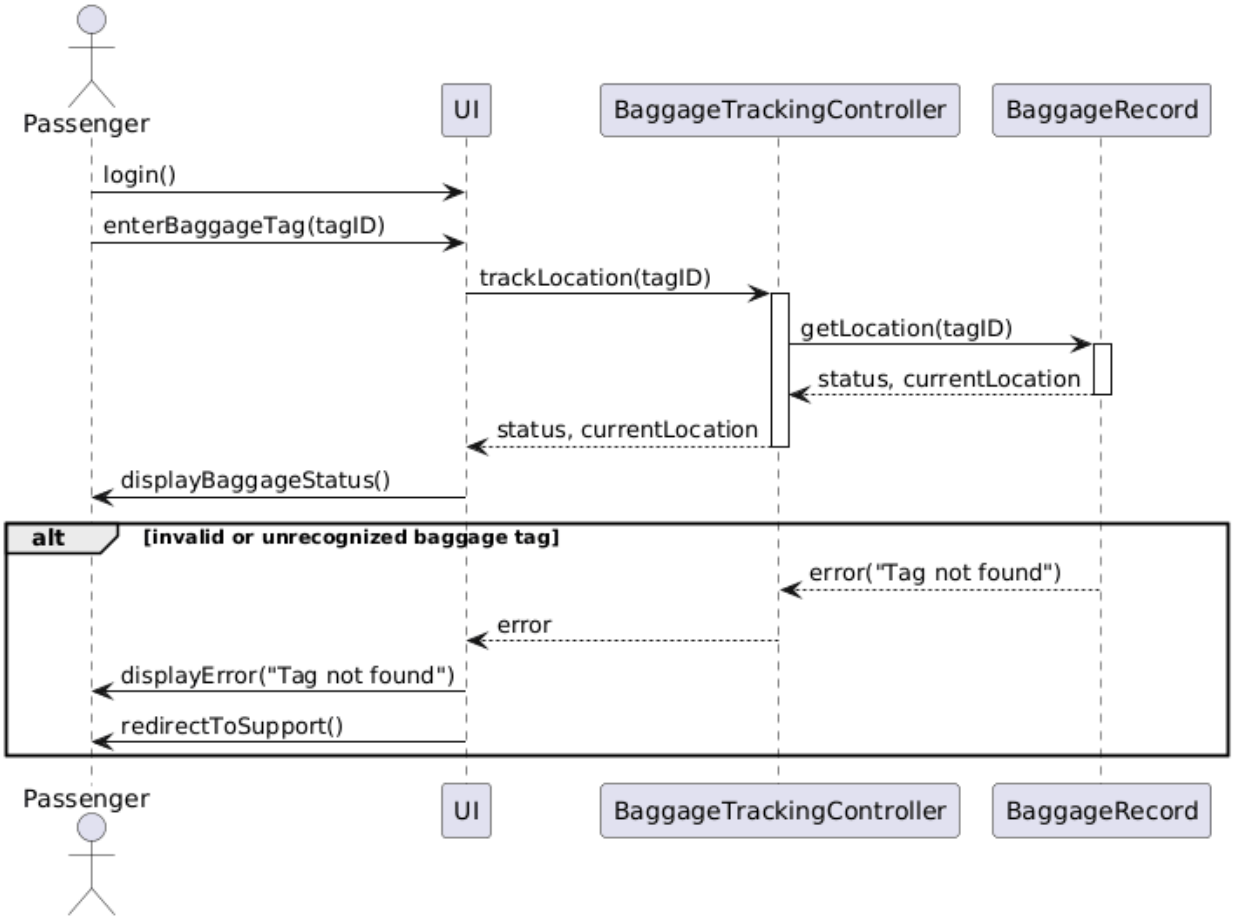
Focus: *Boarding Pass Retrieval and Download*
Description: This sequence demonstrates how a passenger retrieves and downloads their boarding pass after completing check-in.
Key Logic: *The system retrieves the generated BoardingPass and calls downloadPass(format) to produce a file. If check-in has not been completed, the passenger is redirected to UC-03.*
Compliance: *The dependency check on check-in status enforces the correct process order, preventing passengers from downloading a pass before they are formally checked in.*

UC-05: Check Assigned Gate Information



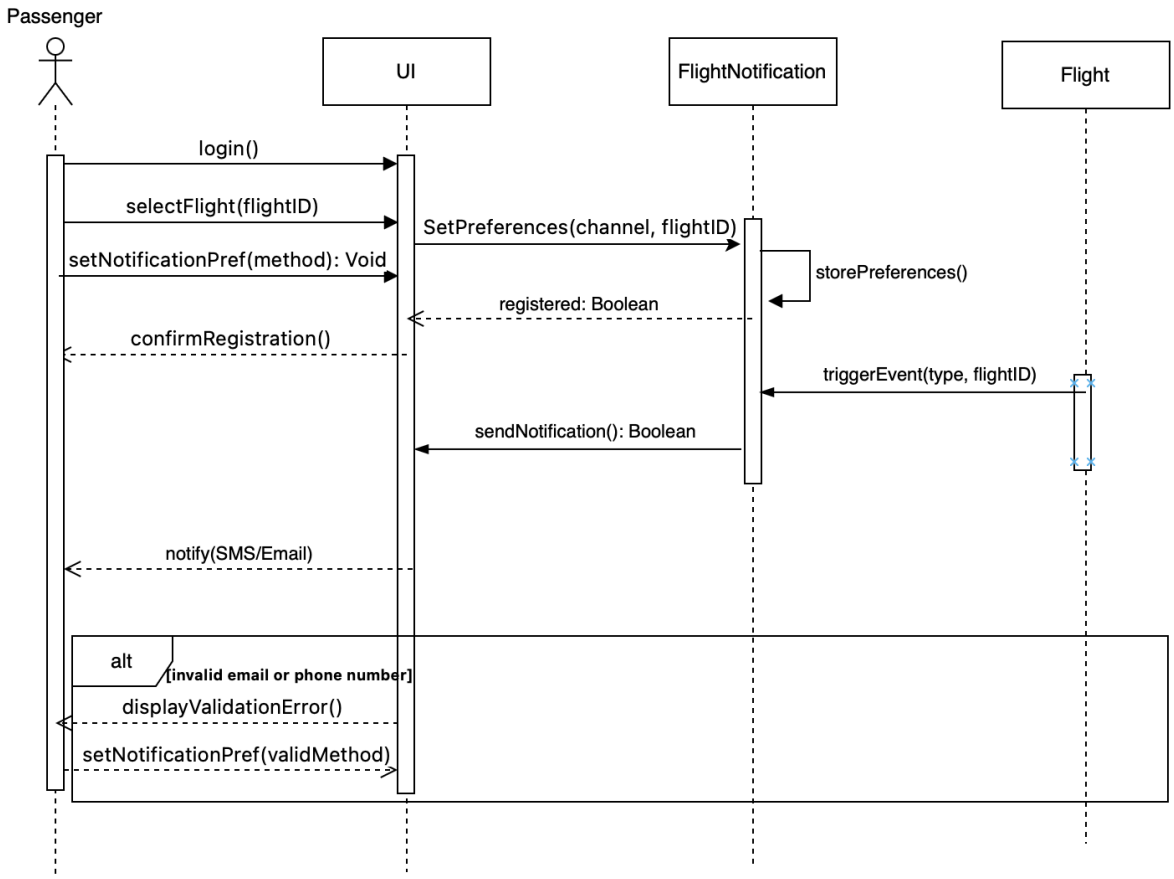
Focus: *Gate Assignment Retrieval*
Description: *This sequence demonstrates how a passenger looks up their assigned departure gate, with AirportStaffassigning the gate as a precondition before the passenger queries the system.*
Key Logic: *The system retrieves the Flight record, which in turn calls getGateInfo() on Gate to return the gate number and terminal. If no gate has been assigned yet, the passenger is advised to check back or enable notifications.*
Compliance: *The precondition of staff gate assignment ensures gate data is present before the passenger queries, preventing the display of incomplete or misleading information.*

UC-06: Track Baggage Status



Focus: *Real-Time Baggage Tracking*
Description: *This sequence demonstrates how a passenger tracks the current status and location of their checked baggage.*
Key Logic: *The system calls getLocation() on BaggageRecord to retrieve the real-time status and current location. If the baggage tag is invalid or unrecognized, an error is displayed and the passenger is redirected to customer support.*
Compliance: *Tag validation at the controller level prevents unrecognized queries from returning empty or incorrect results, ensuring passengers receive accurate tracking information.*

UC-07: Receive Flight Notifications



Postcondition: passenger receives real-time flight alerts

Focus: *Notification Preference Registration and Alert Delivery*
Description: *This sequence demonstrates how a passenger registers for real-time flight alerts and how the system delivers notifications when a flight event occurs.*
Key Logic: *The passenger sets their preferred channel via setNotificationPref(), which the system stores through FlightNotification. When Flight triggers an event (delay, gate change, boarding), FlightNotification calls sendNotification() to deliver the alert via SMS or email.*
Compliance: *Contact validation ensures notifications are only registered with reachable addresses, and the event-driven trigger model guarantees timely delivery without manual intervention.*

Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: *Ani Velo*

Course: *Software Modeling and Design*

Date: *April 10, 2026*

Document Objective

The primary objective of this document is to detail the dynamic behavior of the ***Security Management Module*** *through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill security and safety requirements.*

Technical Scope

This document focuses on the interaction between:

Actors: *Passengers, Airport Staff, and Security Officers.*

Boundary Objects: *User Interfaces (UIs) such as the Screening Dashboard, Incident Reporting Form, and Lockdown Control Panel.*

Control Objects: *Controllers that orchestrate business logic, such as the SecurityController and AlertManager*

Entity Objects: *The database layer including PassengerProfiles, SecurityLogs, ProhibitedItems, and IncidentReports.*

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

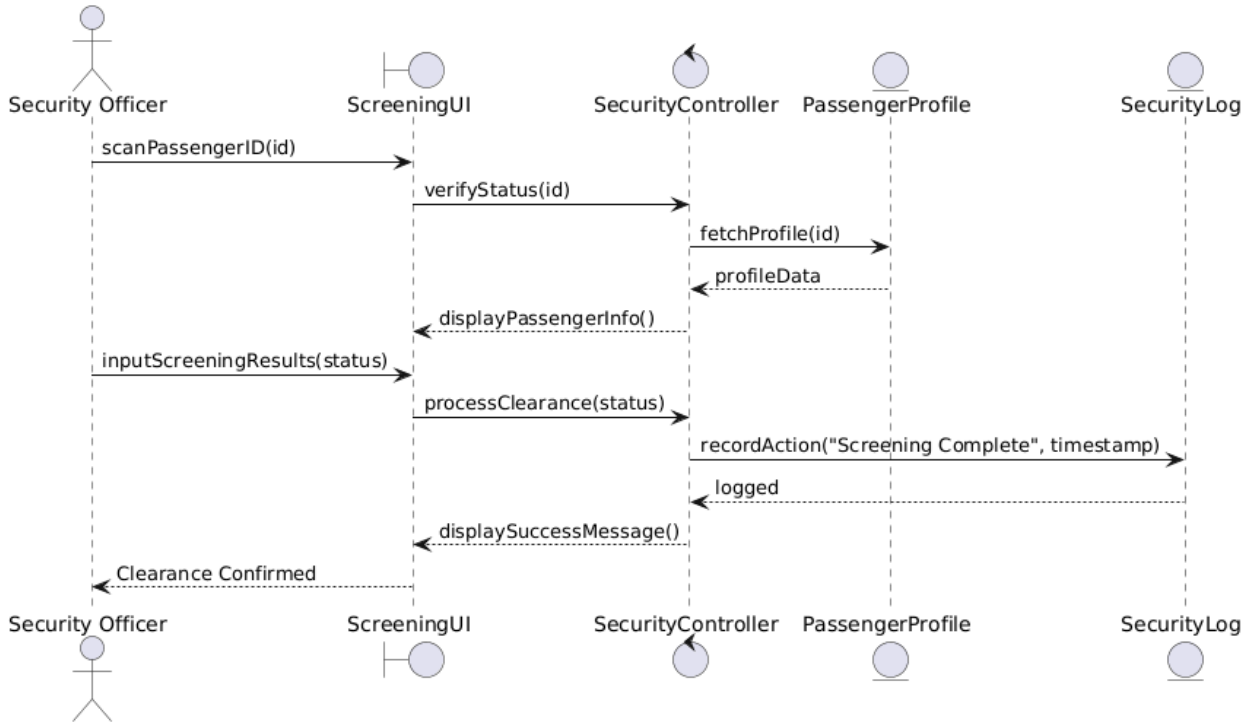
Strict UML Notation: *Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).*

Controller-Centric Logic: *To keep the UI "thin," all business rules and validation steps are localized within "Controller" objects.*

Entity-Layer Persistence: *Direct communication with the database is handled through specific "Entity" objects.*

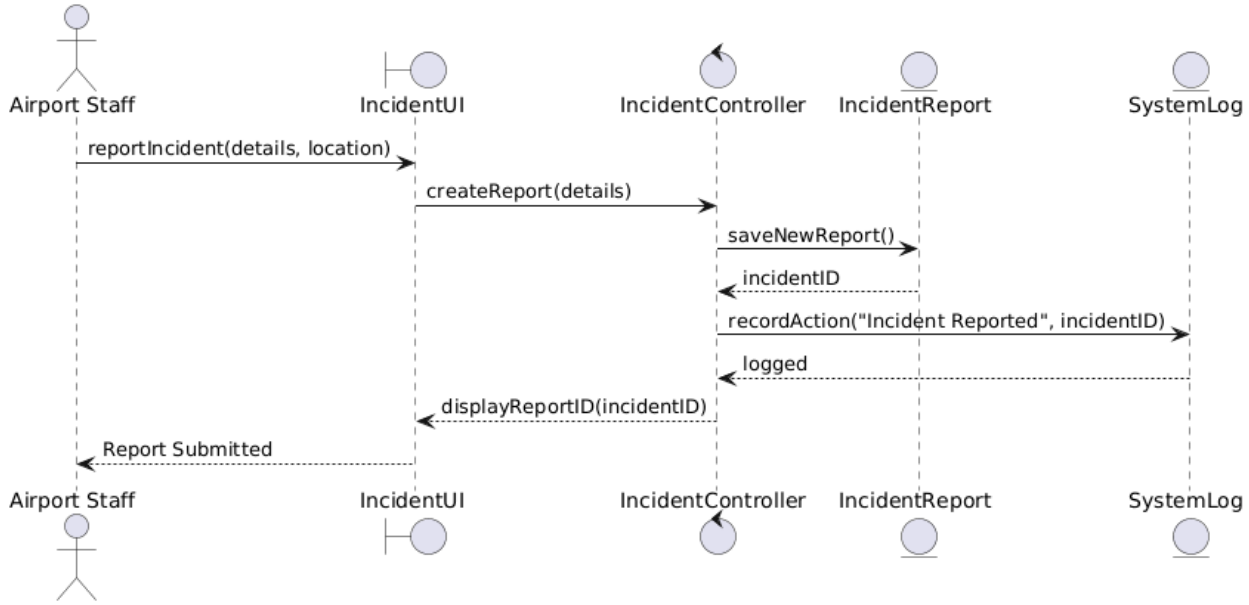
Automated Audit Integration: *Every administrative action that changes the system state includes an automated trigger to the SystemLog entity for auditing purposes.*

1. UC_SEC_01: Passenger Security Screening



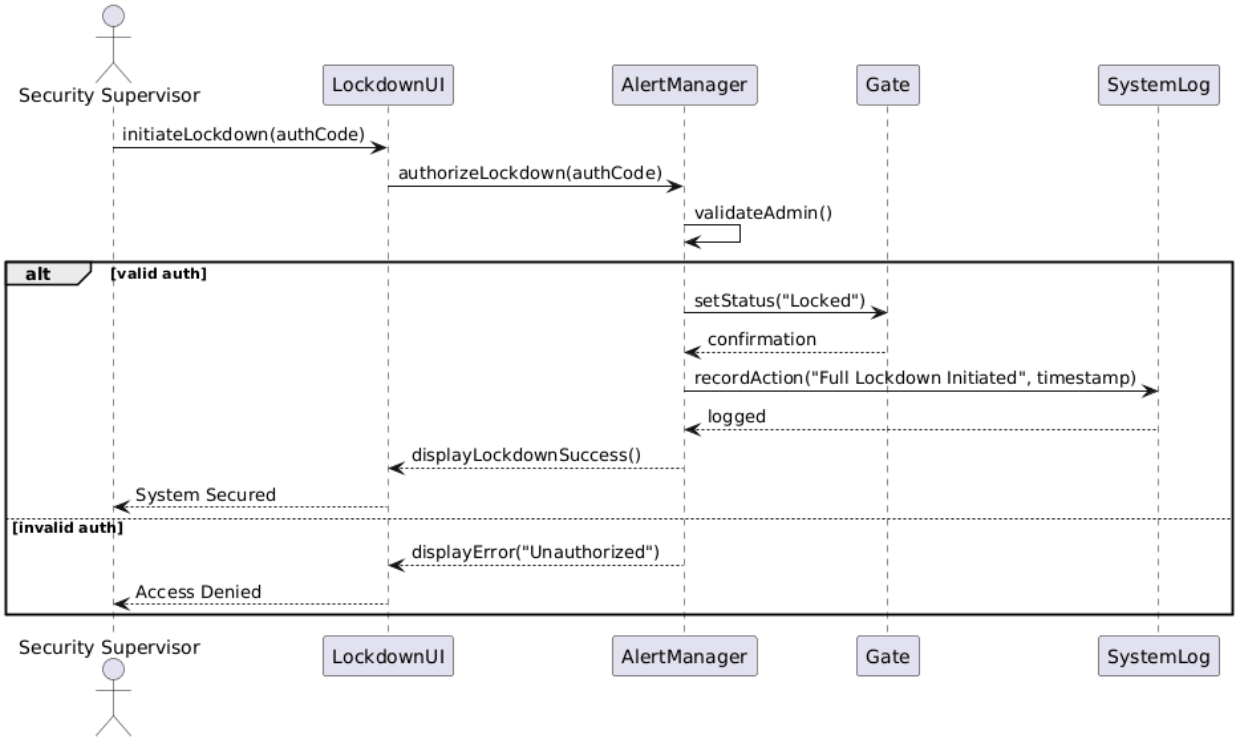
- **Focus:** Screening Workflow and Clearance
- **Description:** This sequence illustrates the process of verifying a passenger's status and logging physical screening results
- **Key Logic:** The SecurityController validates the passenger's ID against the manifest before allowing the officer to enter screening results.
- **Compliance:** The process concludes with a mandatory update to the SecurityLog to ensure all clearances are audited.

2. UC_SEC_02: Report Security Incident



Focus: *Incident Logging and Documentation*
Description: *This sequence demonstrates how a staff member or officer creates a formal security incident report.*
Key Logic: *The system creates a new IncidentReport entity, assigns a unique ID, and updates the status logs.*
Compliance: *Immediate logging in the SystemLog ensures that the discovery of the incident is time-stamped for auditing.*

3. UC_SEC_03: Emergency Facility Lockdown



Focus: *Critical Safety Response*
Description: *This sequence represents the high-priority process of initiating a facility-wide lockdown.*
Key Logic: *The controller verifies administrative permissions and updates the status of physical Gate and Door entities.*
Compliance: *Every maintenance and emergency event is recorded in the audit logs to maintain data integrity.*

Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: *Albina Blloshmi*
Course: *Software Modeling and Design*
Date: *April 2026*

Document Objective

The primary objective of this document is to detail the dynamic behavior of the ***Baggage Management Module*** *through a series of sequence diagrams. While use case diagrams define what the system does, these models illustrate how system components interact over time to support baggage-related operations.*

Technical Scope

This document focuses on the interaction between:

1. ***Actors:*** *Passenger, Airport Staff*
2. ***Boundary Objects:*** *User interfaces used for baggage registration, tracking, and reporting*
3. ***Control Objects:*** *System components responsible for processing requests and managing logic*
4. ***Entity Objects:*** *Core data entities such as Baggage, Flight, and SystemLog*

System Interaction Standards

To ensure consistency, all sequence diagrams follow these principles:

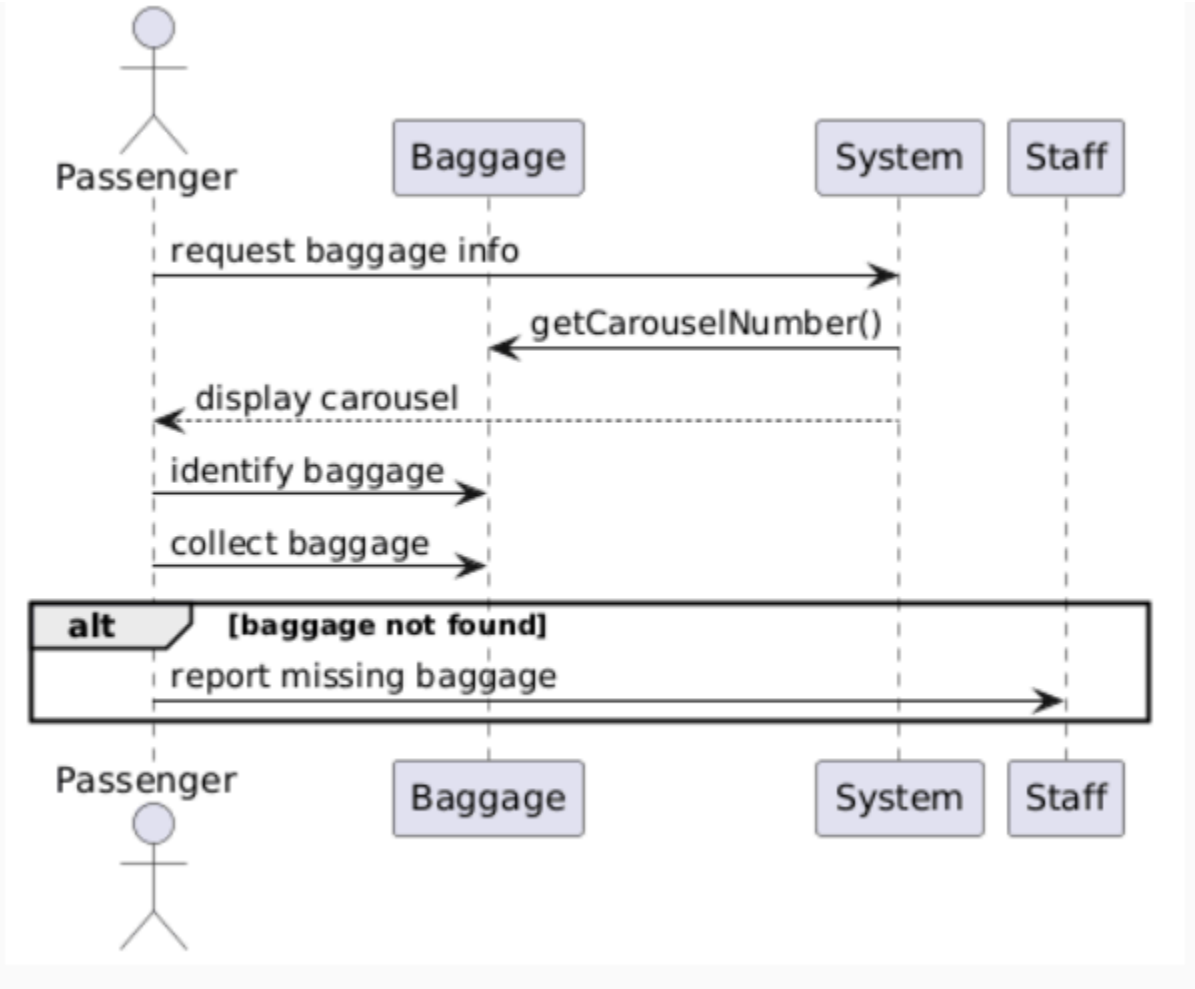
1. ***Standard UML Notation:*** *Solid arrows represent requests, dashed arrows represent responses*
2. ***Controller-Based Logic:*** *Business logic is handled by system/control components*
3. ***Entity Separation:*** *Data handling is performed by entity objects (e.g., Baggage, Flight)*
4. ***Audit Logging:*** *All major actions trigger updates to the SystemLog*
5. ***Main Success Focus:*** *Diagrams emphasize the primary success scenario*

UC-08: Claim Baggage at Destination

Focus: Baggage Retrieval Process

- ***Description:***
  This sequence illustrates how a passenger retrieves baggage upon arrival, including system assistance in locating the correct carousel.

- ***Key Logic:***
  The system provides carousel information and allows the passenger to identify and collect baggage efficiently.
- ***Compliance:***
  Ensures minimal waiting time and clear passenger guidance.

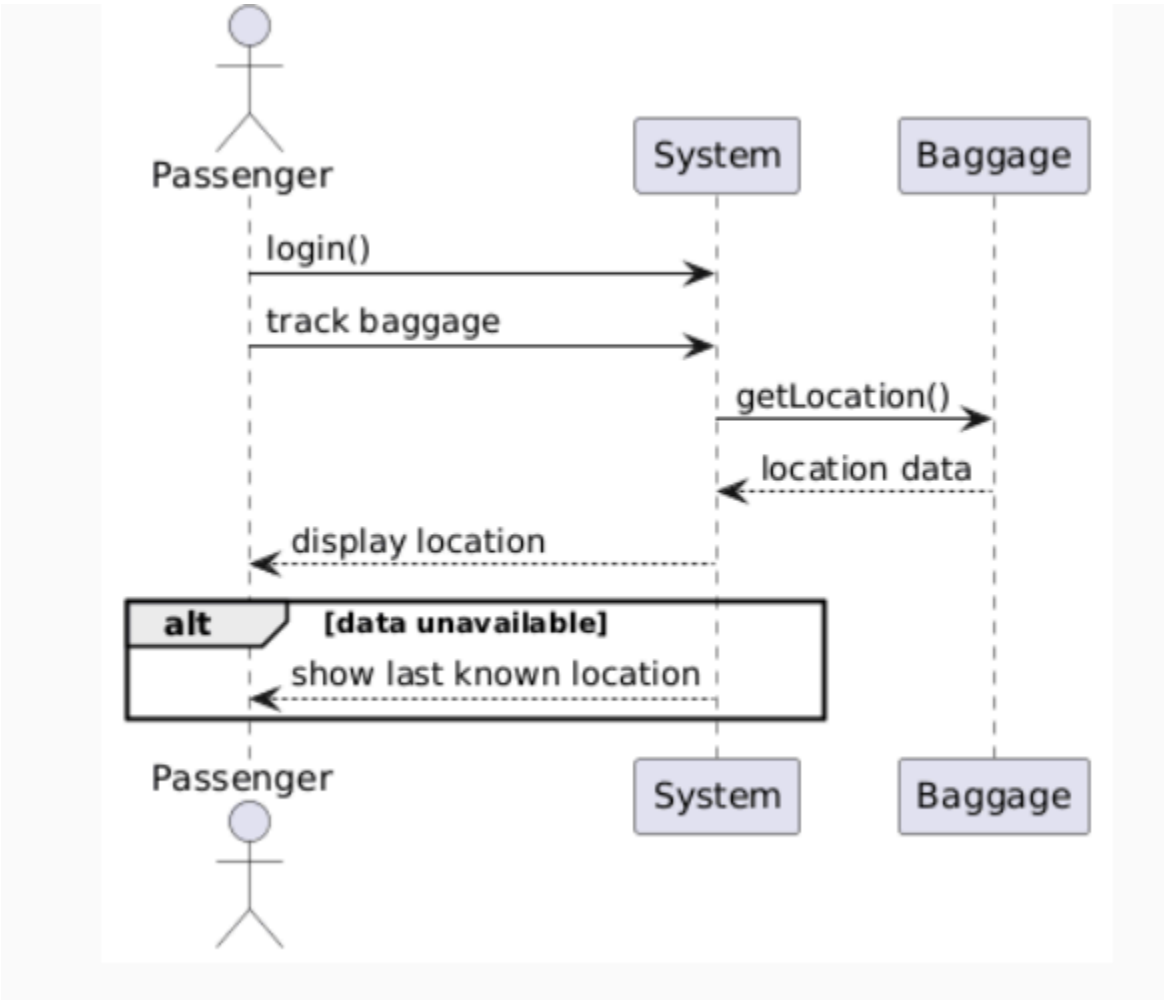


UC-09: Track Baggage in Real-Time

Focus: Real-Time Monitoring

- ***Description:***
  This sequence shows how passengers track baggage location throughout their journey using real-time system updates.

- ***Key Logic:***
  The system retrieves and displays current baggage location data.
- ***Compliance:***
  Ensures high accuracy and real-time responsiveness.

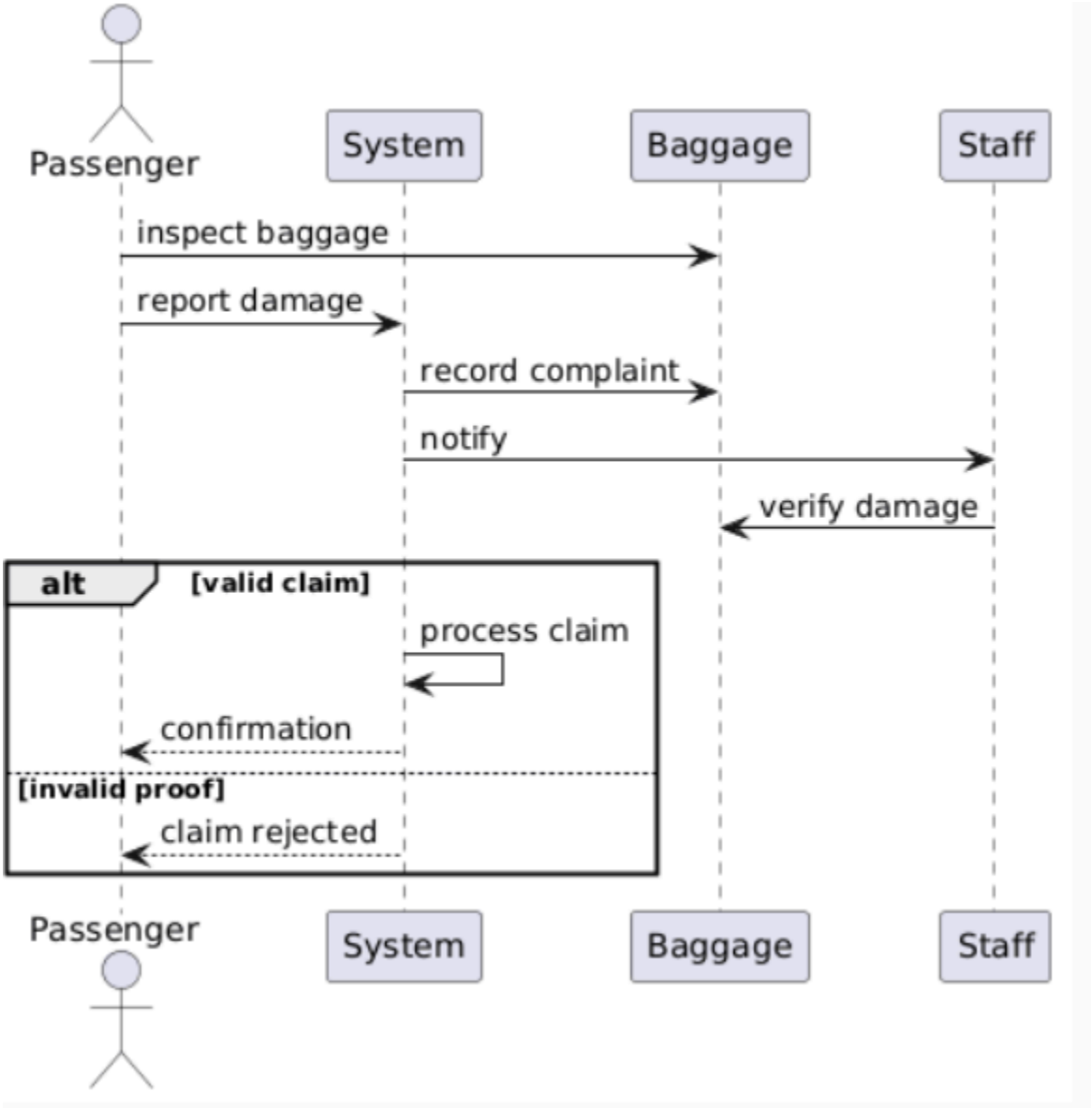


UC-10: Report Damaged Baggage

Focus: Damage Reporting and Claim Handling

- ***Description:***
  This sequence represents how passengers report damaged baggage and how the system processes the claim.

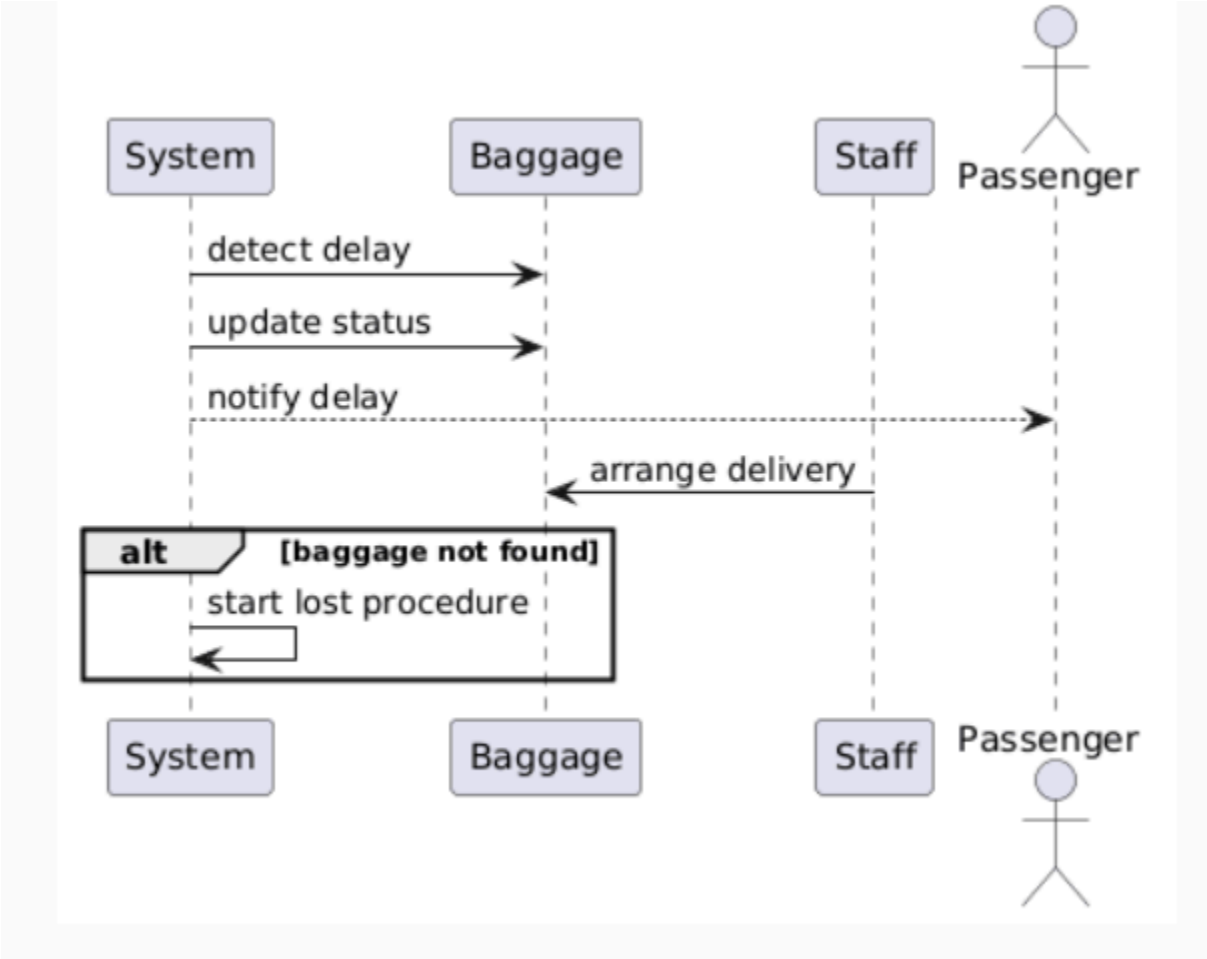
- ***Key Logic:***
  The system records the complaint, staff verifies the damage, and the claim is processed accordingly.
- ***Compliance:***
  Ensures secure and fair claim handling.



UC-11: Handle Delayed Baggage

Focus: Delay Management

- ***Description:***
  This sequence shows how the system detects and handles delayed baggage situations.
- ***Key Logic:***
  The system updates baggage status and notifies passengers, while staff arranges delivery.
- ***Compliance:***
  Ensures timely communication and reliable tracking.

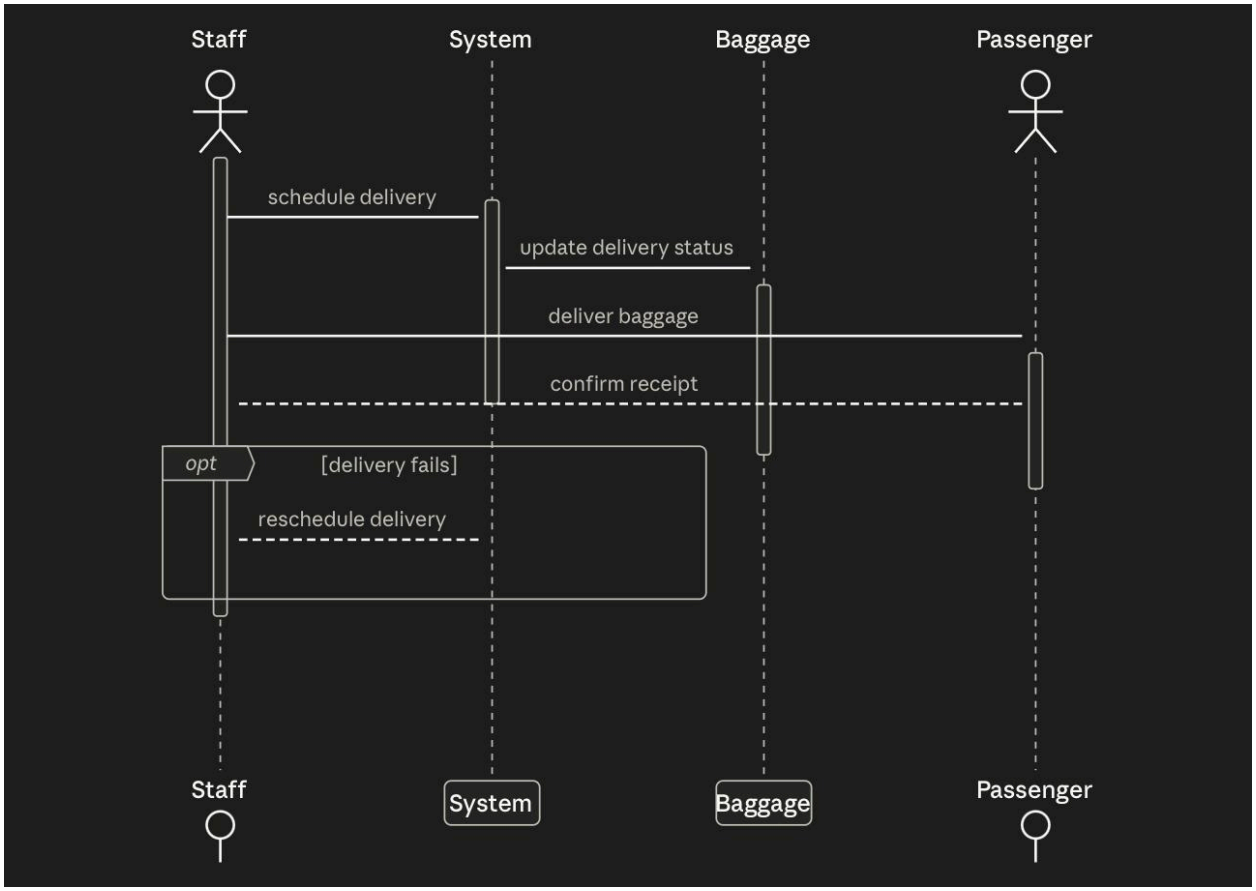


UC-12: Deliver Baggage to Passenger

Focus: Baggage Delivery

- ***Description:***
  This sequence illustrates how delayed or rerouted baggage is delivered to the passenger.
- ***Key Logic:***
  Staff schedules delivery, and the system updates delivery status until confirmation.

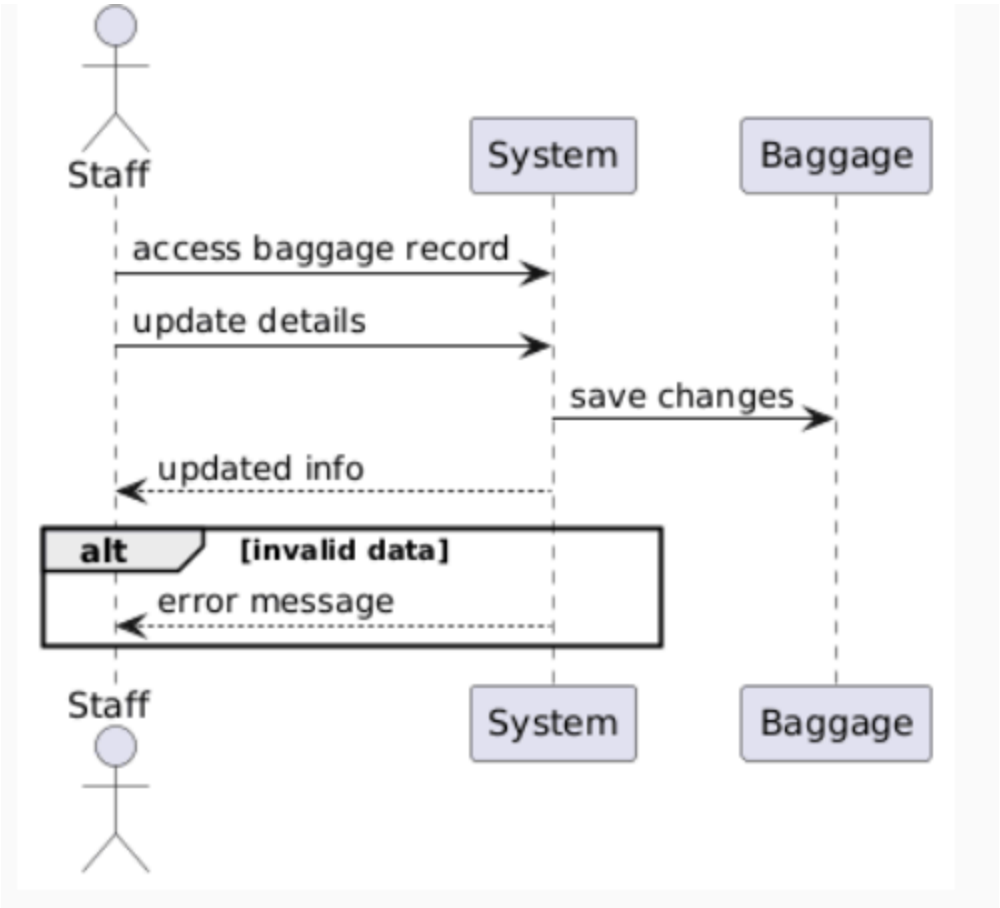
- ***Compliance:***
  Ensures secure and timely delivery.



UC-13: Update Baggage Information

Focus: Data Management

- ***Description:***
  This sequence shows how airport staff update baggage information in the system.
- ***Key Logic:***
  The system validates and saves updated data.
- ***Compliance:***
  Ensures data accuracy and secure access.



AMEO LULAJ:

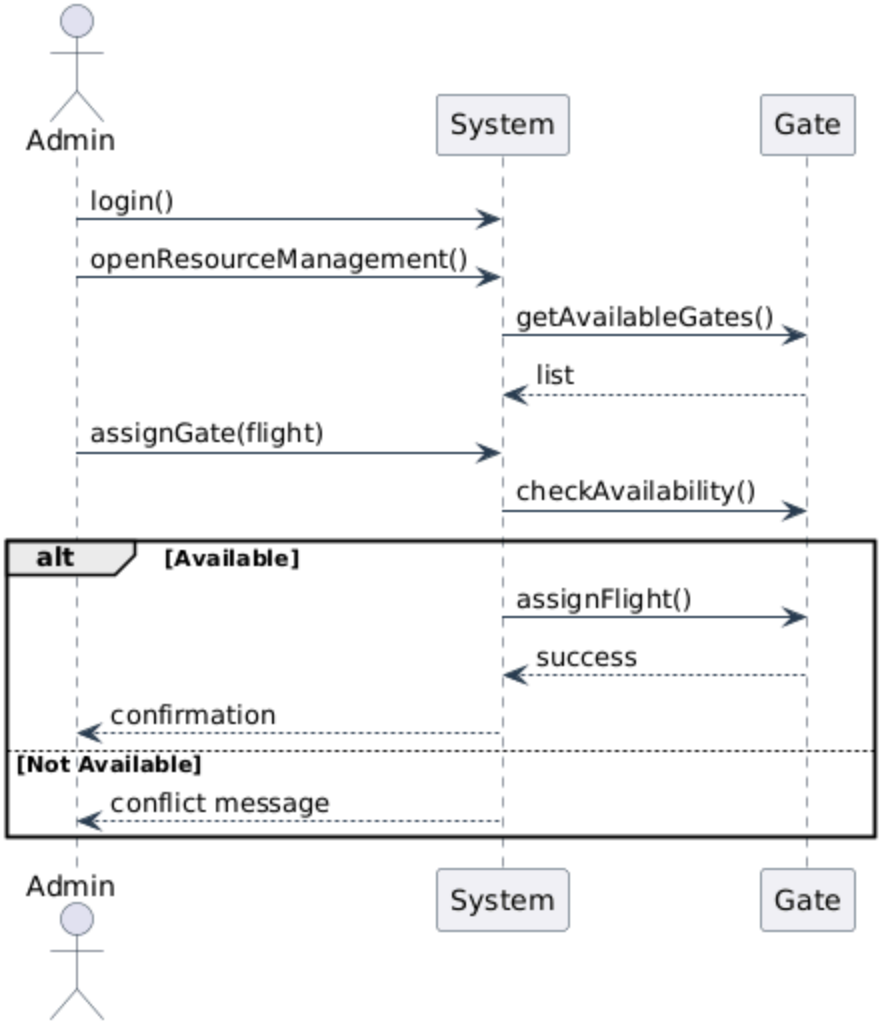
This sequence diagram represents the overall workflow of the Airport Resource Management module by integrating four use cases: Gate Allocation (UC-ARM-01), Runway Scheduling (UC-ARM-02), Staff Allocation (UC-ARM-03), and Equipment Management (UC-ARM-04). The process begins with the System Administrator accessing the resource management system.

Depending on the selected operation, the system interacts with different resources such as Gate, Runway, Staff, or Equipment. Each interaction follows a structured sequence where the system retrieves availability or status information, processes the administrator's request, and performs validation checks.

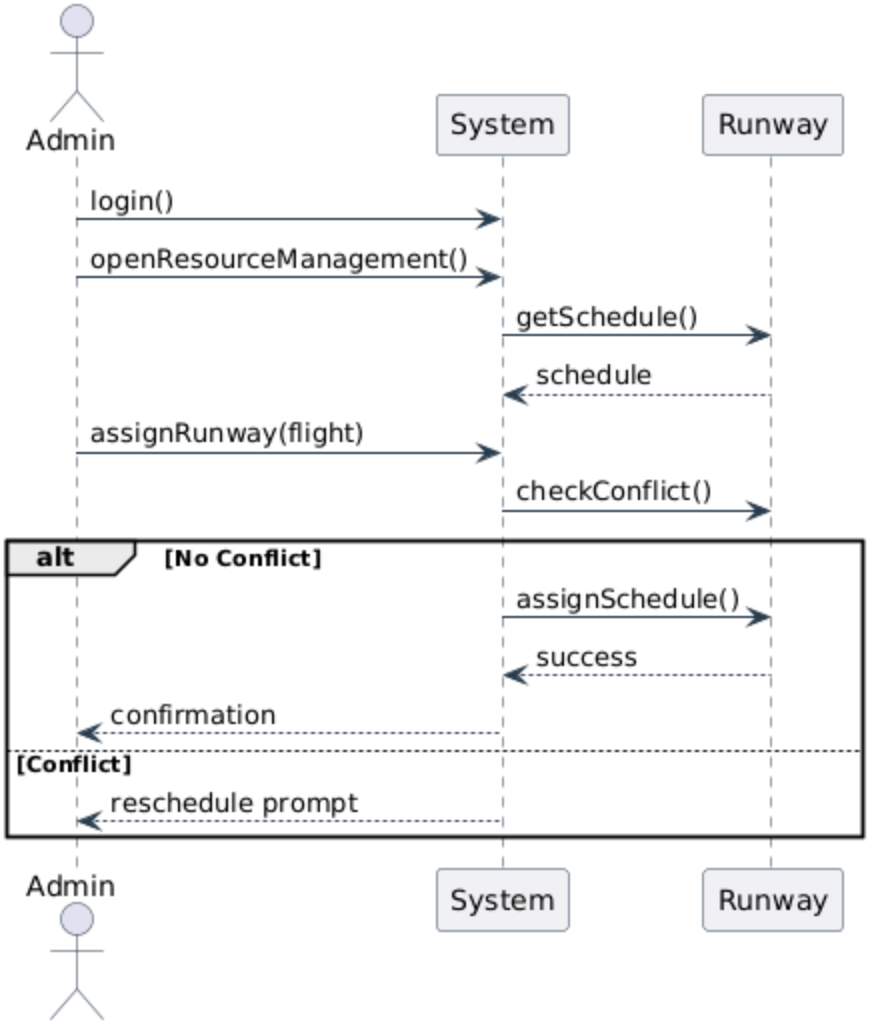
Alternative flows are represented using conditional paths. For example, if a resource is unavailable or a conflict is detected, the system notifies the administrator and suggests corrective actions. Otherwise, the system confirms the successful allocation or update of the resource.

Overall, the diagram demonstrates how the system coordinates multiple resource types while ensuring efficient allocation, conflict handling, and real-time status updates within airport operations.

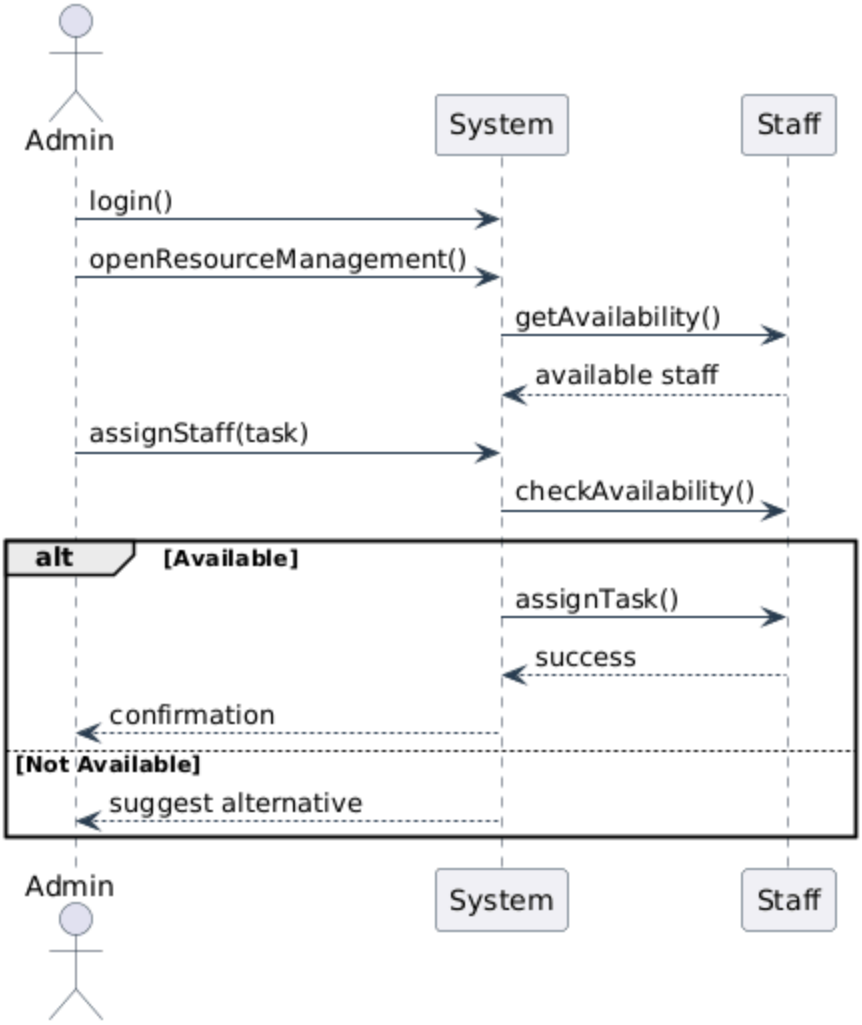
UC-ARM-01: Gate Allocation - Sequence Diagram



UC-ARM-02: Runway Scheduling - Sequence Diagram



UC-ARM-03: Staff Allocation - Sequence Diagram



UC-ARM-04: Equipment Management - Sequence Diagram